



Problem Statement

Scaler is an online tech-versity offering intensive computer science & Data Science courses through live classes delivered by tech leaders and subject matter experts. The meticulously structured program enhances the skills of software professionals by offering a modern curriculum with exposure to the latest technologies. It is a product by InterviewBit.

You are working as a data scientist with the analytics vertical of Scaler, focused on profiling the best companies and job positions to work for from the Scaler database. You are provided with the information for a segment of learners and tasked to cluster them on the basis of their job profile, company, and other features. Ideally, these clusters should have similar characteristics.

Data Dictionary:

'Unnamed 0' - Index of the dataset

Email_hash - Anonymised Personal Identifiable Information (PII)

Company_hash - This represents an anonymized identifier for the company, which is the current employer of the learner.

orgyear - Employment start date

CTC - Current CTC

Job_position - Job profile in the company

CTC_updated_year - Year in which CTC got updated (Yearly increments, Promotions)

Concept Used:

Manual Clustering

Unsupervised Clustering - K- means, Hierarchical Clustering

Importing Libraries

```
In [138... # libraries to analyze data
import numpy as np
import pandas as pd

# libraries to visualize data
import matplotlib.pyplot as plt
import seaborn as sns

from random import sample
```

```

from numpy.random import uniform

import re

from sklearn.preprocessing import LabelEncoder, OneHotEncoder, StandardScaler
from sklearn.impute import KNNImputer
from sklearn.neighbors import NearestNeighbors
from sklearn.cluster import KMeans, AgglomerativeClustering
from sklearn.metrics import silhouette_score
from sklearn.decomposition import PCA
from sklearn.manifold import TSNE
from yellowbrick.cluster import KElbowVisualizer

import scipy.cluster.hierarchy as sch
from scipy.cluster.hierarchy import dendrogram, linkage, fcluster

import umap

from datetime import datetime

```

Inspecting Data

```

In [40]: df = pd.read_csv('Scaler_kmeans_clustering.csv')
df.head()

```

```

Out[40]:

```

	Unnamed: 0	company_hash	email_hash
0	0	atrgxnnt xzaxv	6de0a4417d18ab14334c3f43397fc13b30c35149d70c05..
1	1	qtrxvzwt xzegwgbb rxbxnta	b0aaf1ac138b53cb6e039ba2c3d6604a250d02d5145c10..
2	2	ojzwnvwnxw vx	4860c670bcd48fb96c02a4b0ae3608ae6fdd98176112e9..
3	3	ngpgutaxv	effdede7a2e7c2af664c8a31d9346385016128d66bbc58..
4	4	qxen sqghu	6ff54e709262f55cb999a1c1db8436cb2055d8f79ab520..

```

In [41]: df.describe()

```

```
Out[41]:
```

	Unnamed: 0	orgyear	ctc	ctc_updated_year
count	79438.000000	79395.000000	7.943800e+04	79438.000000
mean	39766.111609	2015.111481	2.925267e+06	2019.937511
std	22966.750686	28.199260	1.634836e+07	1.271842
min	0.000000	0.000000	2.400000e+01	2015.000000
25%	19871.250000	2014.000000	5.000000e+05	2019.000000
50%	39764.500000	2016.000000	8.000000e+05	2020.000000
75%	59655.750000	2018.000000	1.500000e+06	2021.000000
max	79552.000000	2107.000000	1.000150e+09	2021.000000

Check for:

Unique email hashes (to assess repeat learners)

Null values

Duplicates

```
In [42]: print("Unique values in email_hash")
print(df['email_hash'].nunique())
```

Unique values in email_hash
63529

```
In [43]: print("Null Values")
df.isnull().sum()
```

Null Values

```
Out[43]:
```

	0
Unnamed: 0	0
company_hash	19
email_hash	0
orgyear	43
ctc	0
job_position	23424
ctc_updated_year	0

dtype: int64

```
In [44]: print("Duplicate values")
```

```
df.duplicated().sum()
```

Duplicate values

```
Out[44]: np.int64(0)
```

Clean company and job names using regex

```
In [45]: df['company_hash'] = df['company_hash'].apply(lambda x: re.sub('[^A-Za-z0-9 ]+',  
df['job_position'] = df['job_position'].apply(lambda x: re.sub('[^A-Za-z0-9 ]+',
```

```
In [46]: # After cleaning data
```

```
In [47]: df
```

```
Out[47]:
```

	Unnamed: 0	company_hash	email
0	0	atrgxnnt xzaxv	6de0a4417d18ab14334c3f43397fc13b30c35149d70
1	1	qtrxvzwt xzegwgbbrxbxnta	b0aaf1ac138b53cb6e039ba2c3d6604a250d02d5145
2	2	ojzwnvwnxw vx	4860c670bcd48fb96c02a4b0ae3608ae6fdd9817611
3	3	ngpgutaxv	effdede7a2e7c2af664c8a31d9346385016128d66b0
4	4	qxen sqghu	6ff54e709262f55cb999a1c1db8436cb2055d8f79a0
...	...	...	...
79433	79548	xoqg xzaxvz ouvwt qtotvqwy gqsvzxkvnxgz	5aec9333c62e6170c6534926f1e404b4a5a4afc51ec
79434	79549	xzegojo	a2000f4fb51d61eaadafd9ae584a91fb9a236750549
79435	79550	mqvbnv	2541c4f9b4cd39ff1faa03fb01f149bc03cb22c056
79436	79551	qvphntz xzaxv	03dcf532e73feff37471c44ed1710f5cdebeab8f44
79437	79552	ctonta exzvzwt	2c55c5ffb21d0394312ec7acf6b7cd007329e2679a0

79438 rows × 7 columns

```
In [48]: df.info()
```

```

<class 'pandas.core.frame.DataFrame'>
RangeIndex: 79438 entries, 0 to 79437
Data columns (total 7 columns):
#   Column                Non-Null Count  Dtype
---  -
0   Unnamed: 0             79438 non-null  int64
1   company_hash           79438 non-null  object
2   email_hash             79438 non-null  object
3   orgyear                79395 non-null  float64
4   ctc                    79438 non-null  int64
5   job_position           79438 non-null  object
6   ctc_updated_year       79438 non-null  float64
dtypes: float64(2), int64(2), object(3)
memory usage: 4.2+ MB

```

Data Dictionary:

'Unnamed 0' - Index of the dataset

email_hash - Anonymised Personal Identifiable Information (PII)

company_hash - This represents an anonymized identifier for the company, which is the current employer of the learner.

orgyear - Employment start date

ctc - Current CTC

job_position - Job profile in the company

ctc_updated_year - Year in which CTC got updated (Yearly increments, Promotions)

```
In [49]: df.describe(exclude=['float64', 'int64']).T
```

```

Out[49]:

```

	count	unique	top
company_hash	79438	17911	nvnv wgzohrnvzwj otqcxwto
email_hash	79438	63529	aacf9473e3cee3e3f7c322e49bb8a6d5346bb05f3ff5bb...
job_position	79438	626	nan

Observations:

There are 79438 rows and 7 columns

Null Values observed in 3 columns, i.e. company_hash, orgyear, job_position

Further analysis can be better performed post EDA, handling null, duplicates and outliers

Data Preprocessing

Data Cleaning

Checking Duplicates and Treatment

Checking Outliers and Treatment using Capping

Checking Null Values and Treatment using KNN imputation for Numerical attributes

Remove Special characters using Regex

Feature Engineering

```
In [50]: df1=df.copy()
```

```
In [51]: # Check for duplicates  
df1.duplicated().any()
```

```
Out[51]: np.False_
```

```
In [54]: len(df[df.duplicated()])
```

```
Out[54]: 0
```

No Duplicate rows observed overall in the dataset

Check Duplicates based on Email_hash and remove them to ensure uniqueness of each learner's data

```
In [52]: combined_duplicates = df[df.duplicated(subset=['email_hash'], keep=False)]  
print(f"Number of duplicate rows based on 'email_hash': {combined_duplicates.s  
  
# Display the duplicate rows  
print(combined_duplicates)
```

Number of duplicate rows based on 'email_hash': 30157

	Unnamed: 0	company_hash \
1	1	qtrxvzwt xzegwgbx rxbxnta
4	4	qxen sqghu
5	5	yvuuxrj hzbvqqxta bvqptnxzs ucn rna
10	10	ztfstz ogenfvqt
11	11	ngdor ntwy
...	...	...
79432	79547	ihvastz
79433	79548	xoqg xzaxvz ouvwt qtotvqwy gqsvzxkvnxgz
79434	79549	xzegojo
79436	79551	qvphntz xzaxv
79437	79552	ctonta exzvzwt

	email_hash	orgyear	ctc \
1	b0aaf1ac138b53cb6e039ba2c3d6604a250d02d5145c10...	2018.0	449999
4	6ff54e709262f55cb999a1c1db8436cb2055d8f79ab520...	2017.0	1400000
5	18f2c4aa2ac9dd3ae8ff74f32d30413f5165565b90d8f2...	2018.0	700000
10	580f6368103ecb3c4f90245af63340e0b8a7a2dc946c1c...	2019.0	420000
11	72c2171a022115d475c8faac306912a4c95f6dd7fdd320...	2016.0	600000
...	...	...	...
79432	6e6ac9712156afad89668605b35ef01646c23f0f7d9637...	2016.0	900000
79433	5aec9333c62e6170c6534926f1e404b4a5a4afc51ec576...	2020.0	350000
79434	a2000f4fb51d61eaadafd9ae584a91fb9a236750549dc7...	2018.0	500000
79436	03dcf532e73feff37471c44ed1710f5cdebeab8f440ccf...	2015.0	2000000
79437	2c55c5ffb21d0394312ec7acf6b7cd007329e2679a5c82...	2016.0	500000

	job_position	ctc_updated_year
1	FullStack Engineer	2019.0
4	FullStack Engineer	2019.0
5	FullStack Engineer	2020.0
10	FullStack Engineer	2020.0
11	iOS Engineer	2021.0
...	...	...
79432	Backend Engineer	2021.0
79433	nan	2021.0
79434	nan	2021.0
79436	SDET	2021.0
79437	FullStack Engineer	2021.0

[30157 rows x 7 columns]

❖ Total Duplicate Entries: 30,157 rows have non-unique email_hash values.

❖ These duplicates suggest that several learners have multiple entries in the dataset

```
In [56]: df1= df1.drop_duplicates(subset=['email_hash'], keep='last')
```

```
In [57]: df1['email_hash'].duplicated().any()
```

```
Out[57]: np.False_
```

```
In [58]: df1[df1.duplicated(subset=['email_hash'], keep=False)]
```

```
Out[58]:   Unnamed: 0  company_hash  email_hash  orgyear  ctc  job_position  ctc_updated
```

Removed Duplicates based on Email_hash

Multiple individuals can be associated with the same company, so using company_hash alone may not ensure the uniqueness of learners

Could have considered both Email and Company for duplicate. This would have ensured that we don't remove valid records where the same individual is associated with multiple companies. But with limited information currently going ahead with removing duplicates based on Email_hash to ensure uniqueness of each learner's data

```
In [59]: df1.isna().sum()
```

```
Out[59]:
```

	0
Unnamed: 0	0
company_hash	0
email_hash	0
orgyear	43
ctc	0
job_position	0
ctc_updated_year	0

dtype: int64

```
In [61]: def clean_orgyear_column(df, year_col='orgyear', min_year=1900, max_year=2025,
    """
    Cleans and normalizes a year column by:
    - Removing invalid years
    - Converting to datetime
    - Handling missing values
    - Ensuring consistent type (nullable Int64)

    Parameters:
    - df: pandas DataFrame
    - year_col: name of the column to clean
    - min_year, max_year: valid range for years
    - fill_strategy: 'median', 'mode', 'drop', or 'none'

    Returns:
```

```

- df: DataFrame with cleaned year column
"""

# Step 1: Define valid year range
valid_years = set(range(min_year, max_year + 1))

# Step 2: Replace invalid values with NaN
df[year_col] = df[year_col].apply(lambda x: x if x in valid_years else np.nan)

# Step 3: Convert to datetime and extract year again
df[year_col] = pd.to_datetime(df[year_col].dropna().astype(int), format='%Y')
df[year_col] = df[year_col].dt.year

# Step 4: Handle missing values
if fill_strategy == 'median':
    fill_value = int(df[year_col].median())
    df[year_col].fillna(fill_value, inplace=True)
elif fill_strategy == 'mode':
    fill_value = int(df[year_col].mode()[0])
    df[year_col].fillna(fill_value, inplace=True)
elif fill_strategy == 'drop':
    df.dropna(subset=[year_col], inplace=True)
# else 'none' → keep NaNs

# Step 5: Final type casting
df[year_col] = df[year_col].astype('Int64')

return df

```

```

In [62]: # Clean the 'orgyear' column in df1
df1 = clean_orgyear_column(df1, year_col='orgyear', fill_strategy='median')

```

/tmp/ipython-input-2041639693.py:32: FutureWarning: A value is trying to be set on a copy of a DataFrame or Series through chained assignment using an inplace method.

The behavior will change in pandas 3.0. This inplace method will never work because the intermediate object on which we are setting values always behaves as a copy.

For example, when doing 'df[col].method(value, inplace=True)', try using 'df.method({col: value}, inplace=True)' or 'df[col] = df[col].method(value)' instead, to perform the operation inplace on the original object.

```
df[year_col].fillna(fill_value, inplace=True)
```

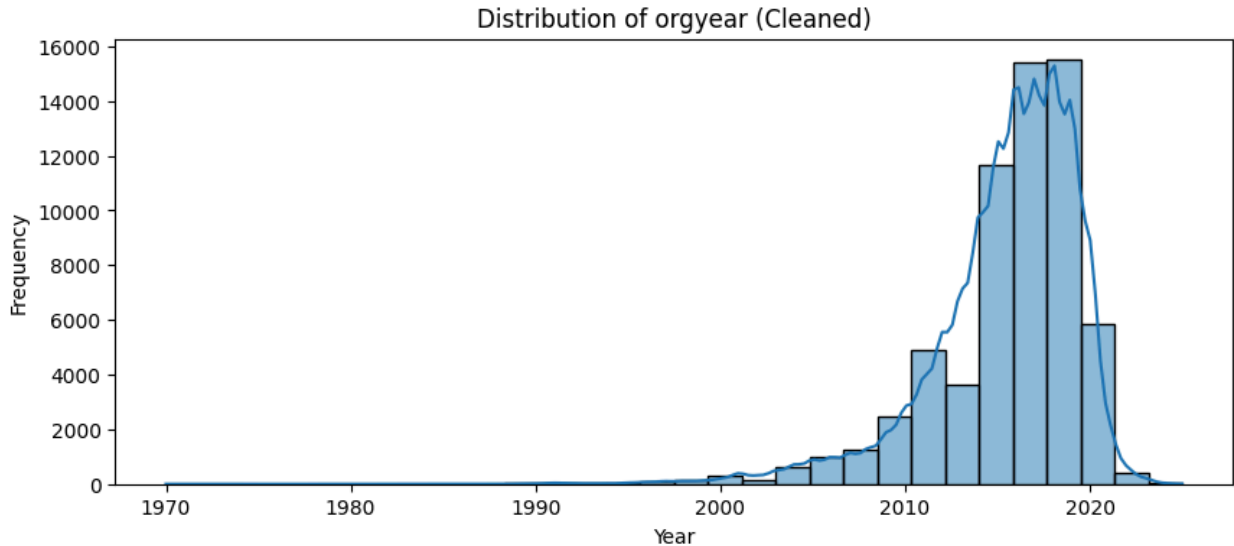
```

In [64]: # Let's visualize the clean data

plt.figure(figsize=(10,4))
sns.histplot(df1['orgyear'].dropna(), bins=30, kde=True)
plt.title("Distribution of orgyear (Cleaned)")
plt.xlabel("Year")
plt.ylabel("Frequency")

```

```
plt.show()
```



```
In [65]: def summarize_columns(df):
    for col in df.columns:
        unique_count = df[col].nunique()
        print(f"\n{'-'*60}")
        print(f"Column: {col}")
        print(f>Data Type: {df[col].dtype}")
        print(f"Total Unique Values: {unique_count}")

        # For columns with too many unique values, limit value_counts display
        max_display = 10

        if pd.api.types.is_numeric_dtype(df[col]):
            print(f"Value counts (normalized):")
            print(df[col].value_counts(normalize=True).head(max_display))
        else:
            print(f"Value counts:")
            print(df[col].value_counts().head(max_display))

        if unique_count > max_display:
            print(f"... (showing top {max_display} of {unique_count} unique values)")
        print(f"\n{'-'*60}")

    # Usage
    summarize_columns(df1)
```

```
-----
Column: Unnamed: 0
Data Type: int64
Total Unique Values: 63529
Value counts (normalized):
Unnamed: 0
79552    0.000016
0         0.000016
2         0.000016
3         0.000016
6         0.000016
7         0.000016
8         0.000016
9         0.000016
13        0.000016
79535    0.000016
Name: proportion, dtype: float64
... (showing top 10 of 63529 unique values)
```

```
-----
Column: company_hash
Data Type: object
Total Unique Values: 17834
Value counts:
company_hash
nvnv wgzohrnvwj otqcxwto    3124
xzegojo                    1969
wgszxkvzn                   1312
vwwtznhq                    1171
zgn vuurxwvmrt vwwghzn     1160
fxuqg rxbxnta               1027
vbvkgz                      940
wvustbxzx                   727
gqvwrt                      692
ywr ntwyzgrgsxto           542
Name: count, dtype: int64
... (showing top 10 of 17834 unique values)
```

```
-----
Column: email_hash
Data Type: object
Total Unique Values: 63529
Value counts:
email_hash
2c55c5fffb21d0394312ec7acf6b7cd007329e2679a5c825c55a45ae509b19e4e    1
6de0a4417d18ab14334c3f43397fc13b30c35149d70c050c0618caea697c87af    1
4860c670bcd48fb96c02a4b0ae3608ae6fdd98176112e90fd66c9df6b37b9059    1
effdede7a2e7c2af664c8a31d9346385016128d66bbc58a44274d5d6876dfec7    1
9bf128ae3f4ea26c7a38b9cdc58cf2acbb8592100c41288e867aac925d40a603    1
756d35a7f6bb8ffeaaffc8fcc9a9ddbb78e7450fa0de2be0a5e9c8d98eb87f97f9    1
e245da546bf50eba09cb7c9976926bd56557d1ac9a17fb019e8de1fdb83fc0d6    1
b2dc928f4c22a9860b4a427efb8ab761e1ce0015fba1a5e804e1dc27e305b06b    1
134cc4a76a119493d523f1855a3b7106f64287455d5cd4bb14f4ad0659fe8a5f    1
6b805c82d8f5de1eab57748e55ceb4e1b774cc1cb7d93ef9137bde42d2a76dcc    1
```

Name: count, dtype: int64
... (showing top 10 of 63529 unique values)

Column: orgyear
Data Type: Int64
Total Unique Values: 46
Value counts (normalized):

orgyear
2018 0.127611
2016 0.121913
2017 0.120968
2019 0.116341
2015 0.102253
2014 0.081128
2020 0.074895
2013 0.056856
2012 0.045601
2011 0.031922

Name: proportion, dtype: Float64
... (showing top 10 of 46 unique values)

Column: ctc
Data Type: int64
Total Unique Values: 1878
Value counts (normalized):

ctc
400000 0.044027
600000 0.042437
500000 0.042060
1000000 0.039667
700000 0.036330
800000 0.034992
300000 0.029341
1200000 0.028428
900000 0.025406
1500000 0.023989

Name: proportion, dtype: float64
... (showing top 10 of 1878 unique values)

Column: job_position
Data Type: object
Total Unique Values: 491
Value counts:

job_position
nan 17896
Backend Engineer 12504
Other 7203
FullStack Engineer 7157
Frontend Engineer 3050
QA Engineer 1970
Support Engineer 1689

```
Engineering Leadership    1657
Android Engineer          1314
Data Analyst              1214
Name: count, dtype: int64
... (showing top 10 of 491 unique values)
```

```
-----
Column: ctc_updated_year
Data Type: float64
Total Unique Values: 7
Value counts (normalized):
ctc_updated_year
2021.0    0.348691
2020.0    0.330290
2019.0    0.208834
2017.0    0.041383
2018.0    0.033622
2016.0    0.024571
2015.0    0.012608
Name: proportion, dtype: float64
```

Data Summary & Observations

1. Unnamed: 0 (int64)

Total Unique Values: 63,529

This column appears to be an index or unique identifier for each row. Since every row has a unique value, it does not carry meaningful categorical information for modeling.

Action: Can be safely dropped during analysis/modeling to avoid redundancy.

2. company_hash (object)

Total Unique Values: 17,834

Distribution is skewed with a few companies having large counts (e.g., top company has 3,124 entries).

Implication: Indicates varying company sizes or learner counts per company.

Action: Useful for grouping learners by company or encoding in modeling. Consider grouping rare companies into an “Other” category to reduce dimensionality.

3. email_hash (object)

Total Unique Values: 63,529 (equal to number of rows)

Every email_hash is unique, which means the dataset is already filtered for uniqueness based on this key.

Implication: This serves as a reliable unique learner identifier.

Action: Use this column for ensuring uniqueness and for joining datasets. No duplicates should exist.

4. orgyear (Int64)

Total Unique Values: 46

Values range mostly from 2011 to 2020, with peak frequencies around 2016-2018.

Implication: This represents a meaningful temporal feature, e.g., the year a learner joined or an event occurred.

Action: Can be used for cohort analysis or time-based trends. Ensure missing values are imputed (median or mode recommended).

5. ctc (int64)

Total Unique Values: 1,878

Salary values cluster around common brackets (e.g., 400,000; 600,000), indicating standardized salary ranges or reporting buckets.

Implication: Reflects compensation trends that can be analyzed or modeled.

Action: Check for outliers and consider normalization or binning if needed.

6. job_position (object)

Total Unique Values: 491

High number of missing values (~17,896 missing), and many distinct job titles.

Most common positions include Backend Engineer, FullStack Engineer, and Other.

Implication: Job role is an important categorical feature but requires cleaning.

Action: Handle missing values by imputation (e.g., 'Unknown'), and consider grouping rare job titles to reduce sparsity.

Removing column 'Unnamed: 0' as it does not have any useful information

```
In [66]: df1 = df1.drop(columns=['Unnamed: 0'])
```

Convert Required Columns to Datetime datatype

```
In [67]: df1['ctc_updated_year'] = pd.to_datetime(df1['ctc_updated_year'], format='%Y')
df1['ctc_updated_year'] = df1['ctc_updated_year'].dt.year
```

Feature Engineering

```
In [70]: # Get the current year
current_year = datetime.now().year

# Create the YOE column by subtracting orgyear from current_year
df1['YOE'] = current_year - df1['orgyear']
```

```
In [71]: df1.head()
```

```
Out[71]:
```

	company_hash	email_hash	orgyear
0	atrgrxnt xzaxv	6de0a4417d18ab14334c3f43397fc13b30c35149d70c05...	2016
2	ojzwnvwnxw vx	4860c670bcd48fb96c02a4b0ae3608ae6fdd98176112e9...	2015
3	ngpgutaxv	effdede7a2e7c2af664c8a31d9346385016128d66bbc58...	2017
6	lubgqsvz wyvot wg	9bf128ae3f4ea26c7a38b9cdc58cf2acbb8592100c4128...	2018
7	vwwtznht ntwyzgrgsj	756d35a7f6bb8ffeaaffc8fcca9dddb78e7450fa0de2be0...	2019

Checking Outliers

```
In [72]: df2=df1.copy()
```

```
In [73]: # Set the style of seaborn
sns.set(style="whitegrid")

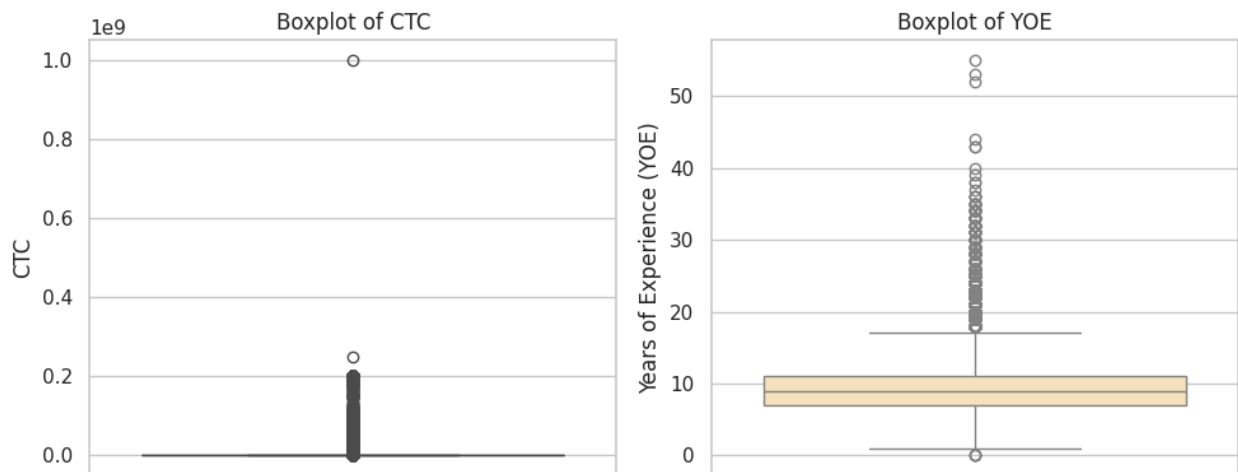
# Create a figure with two subplots for CTC and YOE
fig, ax = plt.subplots(1, 2, figsize=(10, 4))

# Boxplot for CTC
sns.boxplot(data=df2, y='ctc', ax=ax[0])
ax[0].set_title('Boxplot of CTC')
ax[0].set_ylabel('CTC')

# Boxplot for YOE
sns.boxplot(data=df2, y='YOE', ax=ax[1], color='moccasin')
ax[1].set_title('Boxplot of YOE')
ax[1].set_ylabel('Years of Experience (YOE)')

# Show the plots
```

```
plt.tight_layout()
plt.show()
```



We can clearly observe outliers in ctc and YOE

Outliers can significantly impact the performance and results of clustering algorithms like K-means and hierarchical clustering

Outlier Treatment

```
In [74]: def clip_column(df, column, percentile=0.99):
        upper_bound = df[column].quantile(percentile)
        lower_bound = df[column].min()
        df[f'{column}_capped'] = np.clip(df[column], lower_bound, upper_bound)
        return df

# Apply to both 'ctc' and 'YOE'
df2 = clip_column(df2, 'ctc')
df2 = clip_column(df2, 'YOE')
```

```
In [75]: # Set the style of seaborn
sns.set(style="whitegrid")

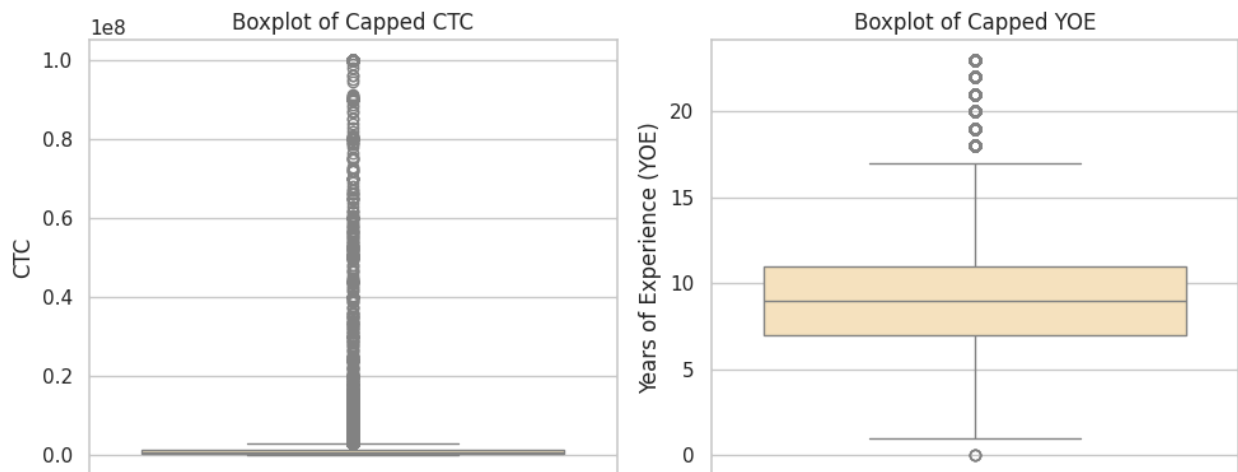
# Create a figure with two subplots for CTC and YOE
fig, ax = plt.subplots(1, 2, figsize=(10, 4))

# Boxplot for CTC post Capping
sns.boxplot(data=df2, y='ctc_capped', ax=ax[0], color='moccasin')
ax[0].set_title('Boxplot of Capped CTC')
ax[0].set_ylabel('CTC')

# Boxplot for YOE post Capping
sns.boxplot(data=df2, y='YOE_capped', ax=ax[1], color='moccasin')
ax[1].set_title('Boxplot of Capped YOE')
ax[1].set_ylabel('Years of Experience (YOE)')

# Show the plots
```

```
plt.tight_layout()
plt.show()
```



We can observe that extreme outliers have been treated with capping method.

Setting the upper percentile to 99% is a way to include most of the data points while excluding the extreme 1% of outliers that are far from the rest of the data.

This approach ensured that the majority of data remains intact, while the extreme values that could significantly impact the clustering results are capped.

Created two new capped columns while keeping the original columns if needed to refer further

Manual Clustering

Creating Designation Flag & Insights

Creating Class Flag & Insights

Creating Tier Flag & Insights

```
In [77]: df3=df2.copy()
# Drop original ctc and YOE columns
df3.drop(['ctc', 'YOE'], axis=1, inplace=True)
```

```
In [78]: df3.info()
```

```
<class 'pandas.core.frame.DataFrame'>
Index: 63529 entries, 0 to 79437
Data columns (total 7 columns):
#   Column                Non-Null Count  Dtype
---  -
0   company_hash           63529 non-null  object
1   email_hash             63529 non-null  object
2   orgyear                63529 non-null  Int64
3   job_position           63529 non-null  object
4   ctc_updated_year       63529 non-null  int32
5   ctc_capped             63529 non-null  int64
6   YOE_capped             63529 non-null  Int64
dtypes: Int64(2), int32(1), int64(1), object(3)
memory usage: 3.8+ MB
```

Creating Flags:

Designation Flag: CTC on the basis of Company, Job Position and Years of Experience

Class Flag: CTC On the basis of Company and Job Position

Tier Flag: CTC On the basis of Company

```
In [79]: import pandas as pd
import numpy as np

# ----- #
# Utility Function: Group & Summarize
# ----- #
def generate_group_summary(df, group_cols, target_col, agg_name):
    """
    Groups the DataFrame based on specified columns and computes summary statistics
    (mean, median, max, min, count) for the target column.

    Returns only the group columns and the computed mean, with a dynamic name.

    Parameters:
    - df: pd.DataFrame
    - group_cols: list of columns to group by
    - target_col: column on which aggregation is applied
    - agg_name: string used to name the mean column for clarity
    """
    summary = (
        df.groupby(group_cols)[target_col]
        .agg(['mean', 'median', 'max', 'min', 'count'])
        .reset_index()
        .rename(columns={'mean': f'mean_{agg_name}'})
    )
    return summary[[*group_cols, f'mean_{agg_name}']]

# ----- #
# Utility Function: Merge Summary
```

```

# ----- #
def merge_group_summary(df, summary_df, on_cols, mean_col):
    """
    Merges the calculated group mean column back to the original DataFrame.

    Parameters:
    - df: original DataFrame
    - summary_df: DataFrame containing the group-wise means
    - on_cols: columns used for merging
    - mean_col: name of the mean column to merge
    """
    return df.merge(summary_df, on=on_cols, how='left')

# ----- #
# Utility Function: Create Flag
# ----- #
def create_flag_column(df, target_col, mean_col, flag_col_name):
    """
    Creates a flag column based on how the target value compares to the group
    - 3 = Above average
    - 2 = Equal to average
    - 1 = Below average

    Parameters:
    - df: input DataFrame
    - target_col: column to compare
    - mean_col: reference column (typically a group mean)
    - flag_col_name: output column for the flag
    """
    df[flag_col_name] = (
        (df[target_col] > df[mean_col]).astype(int) * 2 + # Above mean => 3
        (df[target_col] == df[mean_col]).astype(int)      # Equal to mean =>
    ).replace(0, 1) # Everything else => 1 (below mean)
    return df

# ----- #
# Pipeline: Group, Merge, Flag, Drop
# ----- #
def apply_group_flagging(df, group_cols, target_col, flag_name):
    """
    Main pipeline function to compute a group-wise mean, merge it with the ori
    DataFrame, compare each record to the mean, create a flag column, and clea

    Parameters:
    - df: original DataFrame
    - group_cols: list of columns to group by
    - target_col: column to flag (e.g., 'ctc_capped')
    - flag_name: base name for generated columns (e.g., 'Designation', 'Class')

    Returns:
    - df with an additional flag column (e.g., 'Designation_Flag')
    """
    # Step 1: Generate summary with group mean

```

```

summary_df = generate_group_summary(df, group_cols, target_col, flag_name)

# Step 2: Merge the group mean to the original DataFrame
df = merge_group_summary(df, summary_df, group_cols, f'mean_{flag_name}')

# Step 3: Create the flag column comparing actual vs. group mean
df = create_flag_column(df, target_col, f'mean_{flag_name}', f'{flag_name}')

# Step 4: Drop the intermediate mean column for cleanliness
df.drop(columns=[f'mean_{flag_name}'], inplace=True)

return df

```

```

In [80]: # Apply Designation Flag
# Grouping by company, role, and experience to assign flags based on CTC deviation
df3 = apply_group_flagging(
    df3,
    group_cols=['company_hash', 'job_position', 'YOE_capped'],
    target_col='ctc_capped',
    flag_name='Designation'
)

# Apply Class Flag
# Grouping by company and job position only
df3 = apply_group_flagging(
    df3,
    group_cols=['company_hash', 'job_position'],
    target_col='ctc_capped',
    flag_name='Class'
)

# Apply Tier Flag
# Grouping only by company level (irrespective of position or experience)
df3 = apply_group_flagging(
    df3,
    group_cols=['company_hash'],
    target_col='ctc_capped',
    flag_name='Tier'
)

```

Flag Validation

```

In [81]: # Check if all flag columns are present
print(df3.columns)

# Preview a few rows
df3[['ctc_capped', 'Designation_Flag', 'Class_Flag', 'Tier_Flag']].head()

Index(['company_hash', 'email_hash', 'orgyear', 'job_position',
      'ctc_updated_year', 'ctc_capped', 'YOE_capped', 'Designation_Flag',
      'Class_Flag', 'Tier_Flag'],
      dtype='object')

```

```
Out[81]:
```

	ctc_capped	Designation_Flag	Class_Flag	Tier_Flag
0	1100000	1	2	2
1	2000000	1	1	1
2	700000	1	1	1
3	1500000	1	1	1
4	400000	1	1	1

```
In [83]: # Distribution of each flag
for flag in ['Designation_Flag', 'Class_Flag', 'Tier_Flag']:
    print(f'\n{flag} value counts:')
    print(df3[flag].value_counts(normalize=True).sort_index())
```

Designation_Flag value counts:

Designation_Flag

1 0.843693

2 0.156307

Name: proportion, dtype: float64

Class_Flag value counts:

Class_Flag

1 0.809205

2 0.190795

Name: proportion, dtype: float64

Tier_Flag value counts:

Tier_Flag

1 0.81429

2 0.18571

Name: proportion, dtype: float64

◇ What These Results Mean ◇ Designation_Flag

84.4% of learners have CTC below the average for their company + job_position + YOE

15.6% have CTC exactly equal to the mean

0% above mean (flag = 3 missing)

◇ Class_Flag

Similar to above: majority (80.9%) below their company + job_position average

◇ Tier_Flag

81.4% are below the overall company average, across roles and experience

Checking Consistency Across Similar Groups

```
In [84]: # Mean CTC by group for consistency check
df3.groupby(['job_position', 'YOE_capped'])['ctc_capped'].mean().head(10)
```

```
Out[84]:
```

		ctc_capped	
	job_position	YOE_capped	
		14	700000.0
	7	6	420000.0
		7	470000.0
	857628	4	450000.0
	896651	9	600000.0
	ASSISTANT PROFESSOR	20	600000.0
	Account	13	2000000.0
	Administrative clerk	10	500000.0
	Administrator	18	3500000.0
	Advisory Software Engineer	9	1290000.0

dtype: float64

```
In [85]: fig, axes = plt.subplots(1, 3, figsize=(18, 6))

# Plot the distribution of Designation_Flag
sns.countplot(x='Designation_Flag', data=df3, ax=axes[0], color='moccasin')
axes[0].set_title('Distribution of Designation Flag')
axes[0].set_xlabel('Designation Flag')
axes[0].set_ylabel('Count')

# Plot the distribution of Class_Flag
sns.countplot(x='Class_Flag', data=df3, ax=axes[1], color='moccasin')
axes[1].set_title('Distribution of Class Flag')
axes[1].set_xlabel('Class Flag')
axes[1].set_ylabel('Count')

# Plot the distribution of Tier_Flag
sns.countplot(x='Tier_Flag', data=df3, ax=axes[2], color='moccasin')
axes[2].set_title('Distribution of Tier Flag')
axes[2].set_xlabel('Tier Flag')
axes[2].set_ylabel('Count')

# Display the plots
plt.tight_layout()
plt.show()
```



Discuss the distribution of learners based on the Tier flag:

Which companies dominate in Tier 1 and why might this be the case?

Are there any notable patterns or insights when comparing learners from Tier 3 across different companies?

```
In [86]: # Filter for Tier 1 learners
tier_1_learners = df3[df3['Tier_Flag'] == 1]

# Count learners in Tier 1 for each company
tier_1_company_counts = tier_1_learners['company_hash'].value_counts().reset_i
tier_1_company_counts.columns = ['company_hash', 'count']

# Top companies in Tier 1
top_tier_1_companies = tier_1_company_counts.head(10)
print(top_tier_1_companies)
```

	company_hash	count
0	nvnv wgzohrnvzwj otqcxwto	3026
1	xzegojo	1887
2	wgszxkvzn	1247
3	vwwtznht	1127
4	zgn vuurxwvmrt vwwghzn	1094
5	fxuqg rxbxnta	992
6	vbvkgz	786
7	wvustbxzx	676
8	ywr ntwyzgrgsxto	508
9	gqvwr	467

```
In [87]: # Filter for Tier 3 learners
tier_3_learners = df3[df3['Tier_Flag'] == 3]

# Count learners in Tier 3 for each company
tier_3_company_counts = tier_3_learners['company_hash'].value_counts().reset_i
tier_3_company_counts.columns = ['company_hash', 'count']

# Top companies in Tier 3
top_tier_3_companies = tier_3_company_counts.head(10)
print(top_tier_3_companies)
```

```
Empty DataFrame
Columns: [company_hash, count]
Index: []
```

Summary Statistics

```
In [88]: designation_summary = df3.groupby('Designation_Flag')['ctc_capped'].describe()
class_summary = df3.groupby('Class_Flag')['ctc_capped'].describe()
tier_summary = df3.groupby('Tier_Flag')['ctc_capped'].describe()
```

```
In [89]: designation_summary
```

```
Out[89]:
```

	count	mean	std	min	25%	50%	75%	max
Designation_Flag								
1	53599.0	2.288555e+06	1.050545e+07	24.0	440000.0	750000.0	1100000.0	2100000.0
2	9930.0	4.887414e+06	1.690806e+07	95000.0	800000.0	1250000.0	1700000.0	2100000.0

```
In [90]: class_summary
```

```
Out[90]:
```

	count	mean	std	min	25%	50%	75%	max
Class_Flag								
1	51408.0	1.894014e+06	9.245393e+06	24.0	420000.0	700000.0	1100000.0	2100000.0
2	12121.0	6.090981e+06	1.871165e+07	50000.0	1100000.0	1700000.0	2100000.0	2100000.0

```
In [91]: tier_summary
```

```
Out[91]:
```

	count	mean	std	min	25%	50%	75%	max
Tier_Flag								
1	51731.0	1.552737e+06	7.650341e+06	24.0	420000.0	700000.0	1100000.0	2100000.0
2	11798.0	7.702291e+06	2.143350e+07	15000.0	1250000.0	1981000.0	2100000.0	2100000.0

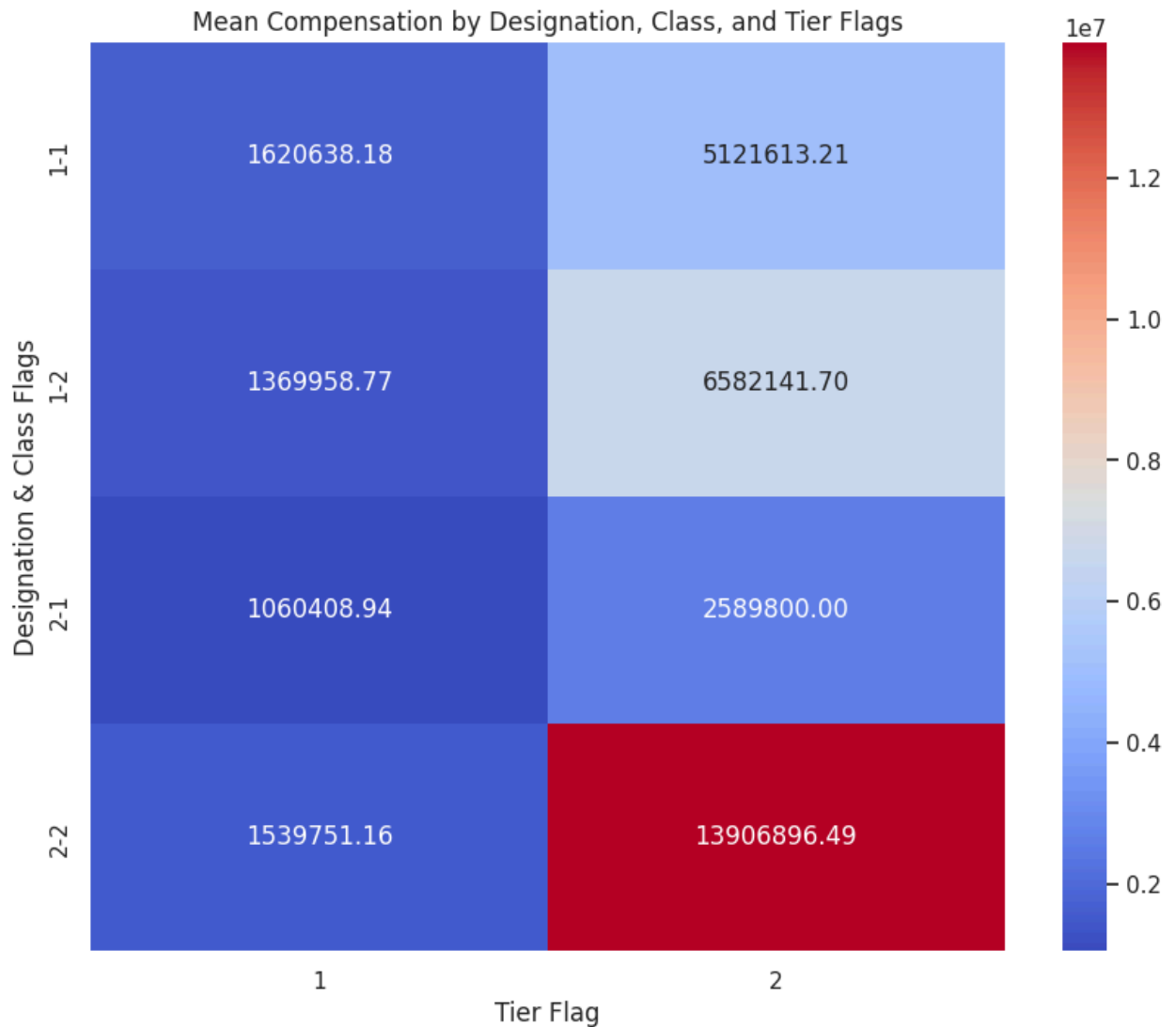
Mean CTC in all the categories and under each flag is similar

Visualizing Mean Compensation

```
In [92]: # Mean compensation by flags
mean_compensation_flags = df3.groupby(['Designation_Flag', 'Class_Flag', 'Tier_Flag'])['ctc_capped'].mean()

# Plot the heatmap
plt.figure(figsize=(10, 8))
sns.heatmap(mean_compensation_flags, annot=True, cmap='coolwarm', fmt='.2f')
plt.title('Mean Compensation by Designation, Class, and Tier Flags')
```

```
plt.xlabel('Tier Flag')
plt.ylabel('Designation & Class Flags')
plt.show()
```



****Cross-Tabulation Analysis: ****

Cross-tabulation (often called a crosstab) is a method to summarize and analyze the relationship between two or more categorical variables by creating a contingency table.

It shows the frequency (count) or other statistics (like percentages) of observations that fall into each combination of categories from the variables.

This helps us watch patterns, associations, or dependencies between variables.

```
In [94]: designation_class_ct = pd.crosstab(df3['Designation_Flag'], df3['Class_Flag'],
print("Cross-Tabulation between Designation Flag and Class Flag:\n", designati
```

Cross-Tabulation between Designation Flag and Class Flag:

Class_Flag	1	2
Designation_Flag		
1	0.869214	0.130786
2	0.485297	0.514703

Insights:

When Designation_Flag = 1 (below average salary in designation grouping), ~87% of these records also have Class_Flag = 1 (below average salary in class grouping), indicating strong alignment at the “below average” level across both grouping criteria.

When Designation_Flag = 2 (equal to mean in designation grouping), the distribution between Class_Flag 1 and 2 is more balanced (~48% vs ~51%). This suggests that being average at the designation level doesn’t guarantee being average at the class level — some variability exists.

Overall, the flags tend to agree mostly when salaries are below average but less so when salaries are around average.

```
In [ ]: designation_tier_ct = pd.crosstab(df3['Designation_Flag'], df3['Tier_Flag'], normalize=True)
print("\nCross-Tabulation between Designation Flag and Tier Flag:\n", designation_tier_ct)
```

Insights:

For Designation_Flag = 1, ~84% are also flagged as Tier_Flag = 1 (below average salary at company-wide level), indicating consistent "below average" salary signals between designation and company tier.

For Designation_Flag = 2, the below-average Tier flag proportion drops to ~69%, showing more variation when employees earn average salary at designation but might be below average at company level.

This suggests that company-wide salary flagging (Tier) is somewhat aligned with designation-level flags but has more tolerance for variation.

```
In [ ]: class_tier_ct = pd.crosstab(df3['Class_Flag'], df3['Tier_Flag'], normalize=True)
print("\nCross-Tabulation between Class Flag and Tier Flag:\n", class_tier_ct)
```

Insights:

The strongest alignment is here: when Class_Flag = 1, a very high proportion (~90%) also has Tier_Flag = 1, reinforcing that below-average salary at class level strongly predicts below-average at company tier level.

For Class_Flag = 2, the distribution flips: ~57% have Tier_Flag = 2 (average or above at tier level), meaning average salaries at class level don't always map to below average at company level.

This implies class-level salary flags correlate strongly with company-wide flags for below-average earners but diverge more for average earners.

Exploratory Data Analysis

UniVariate Analysis

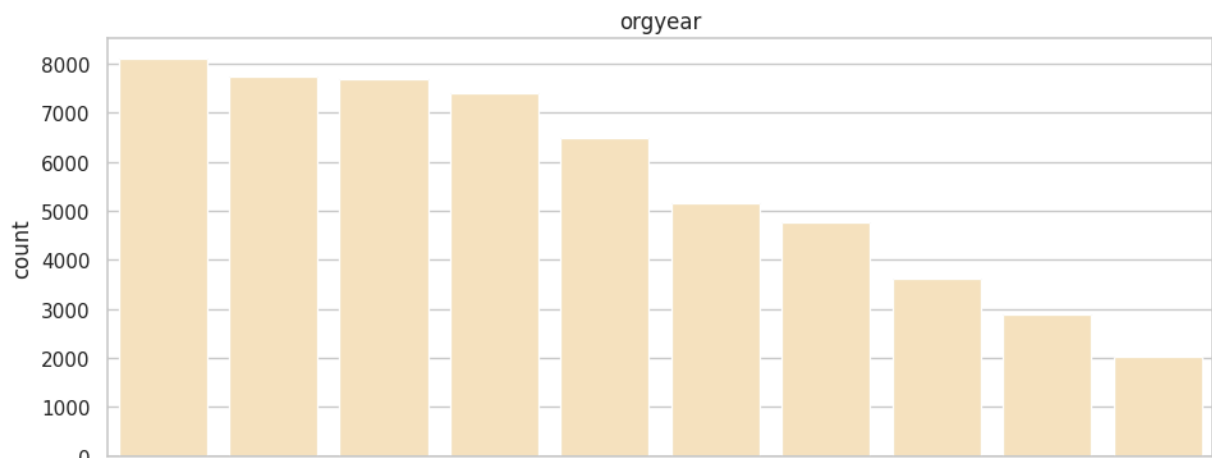
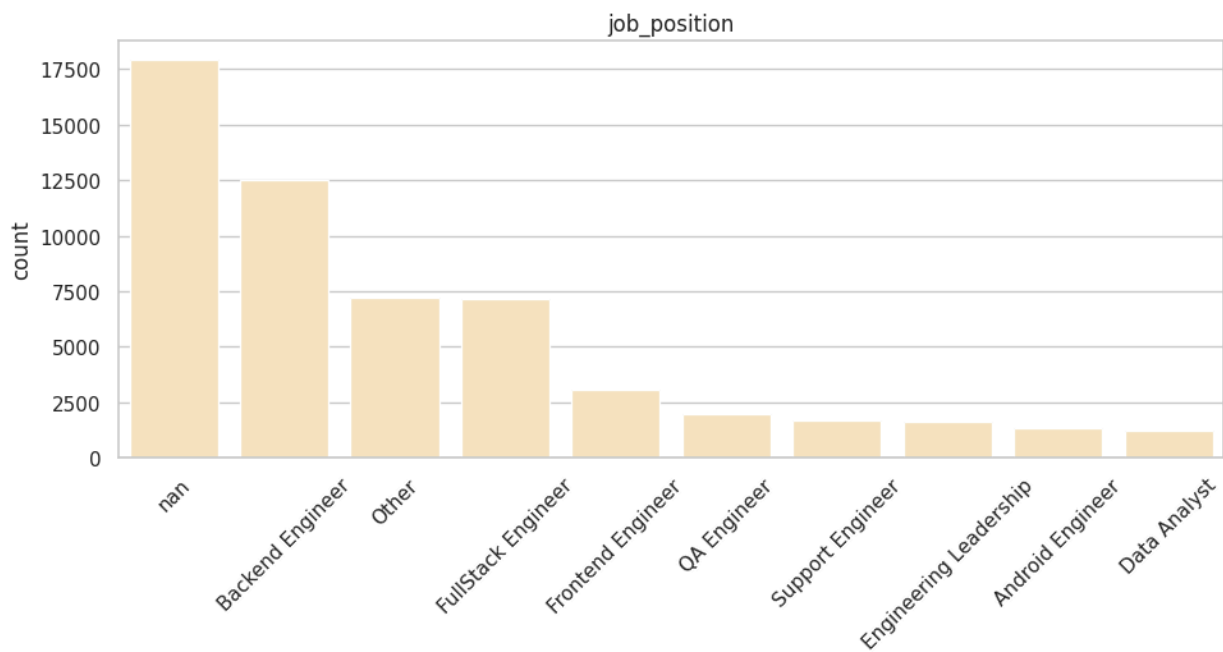
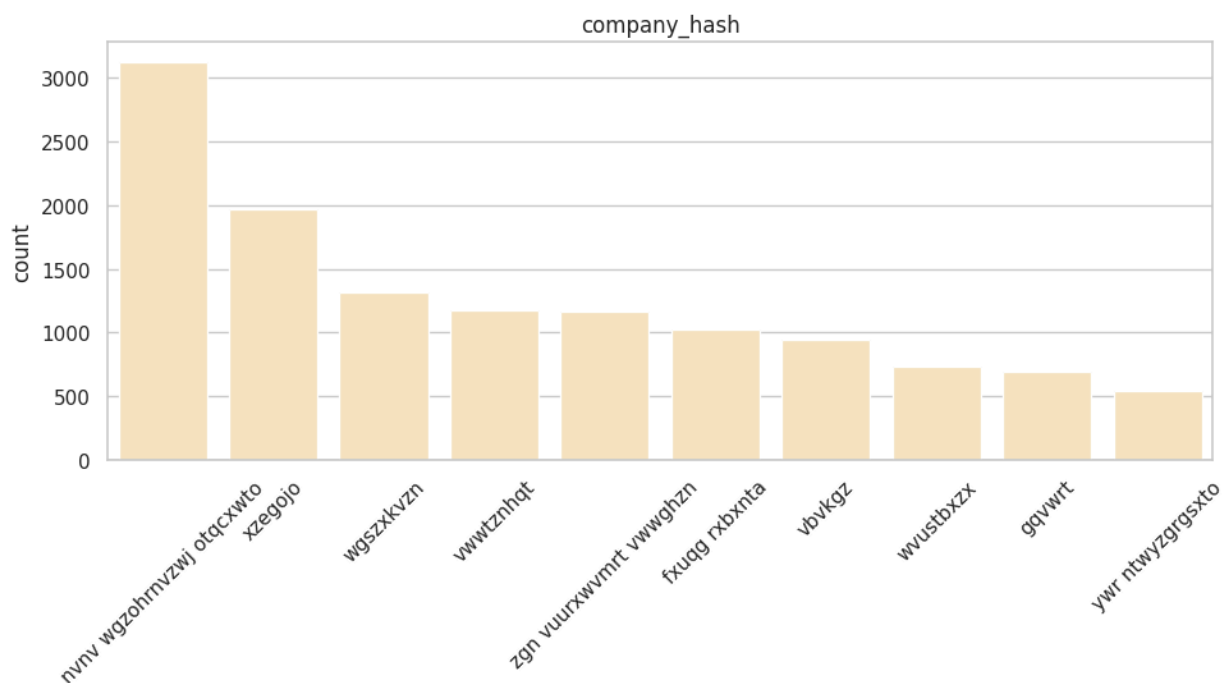
BiVariate Analysis

Statistical Summary

Categorical Feature Distribution

```
In [96]: obj_cols= ['company_hash', 'job_position', 'orgyear']  
  
         num_cols= ['ctc_capped', 'YOE_capped']
```

```
In [97]: plt.figure(figsize=(10, 15))  
  
         i = 1  
         for col in obj_cols:  
             # Get the top 10 values for the column  
             top_10 = df3[col].value_counts().nlargest(10)  
             top_10_index = top_10.index  
  
             ax = plt.subplot(3, 1, i)  
             sns.countplot(x=df3[col], order=top_10_index, color='moccasin')  
             plt.title(f'{col}')  
             if i <= 2:  
                 plt.xticks(rotation=45)  
  
             ax.set_xlabel('')  
             i += 1  
  
         plt.tight_layout()  
         plt.show()
```



Highlights:

We can easily find top 10 companies in terms of count in the dataset Top job position is 'unknown' followed by 'Backend Engineer' and 'FullStack Engineer'

Most of the employees started working in the year 2016 followed by 2018 and 2017

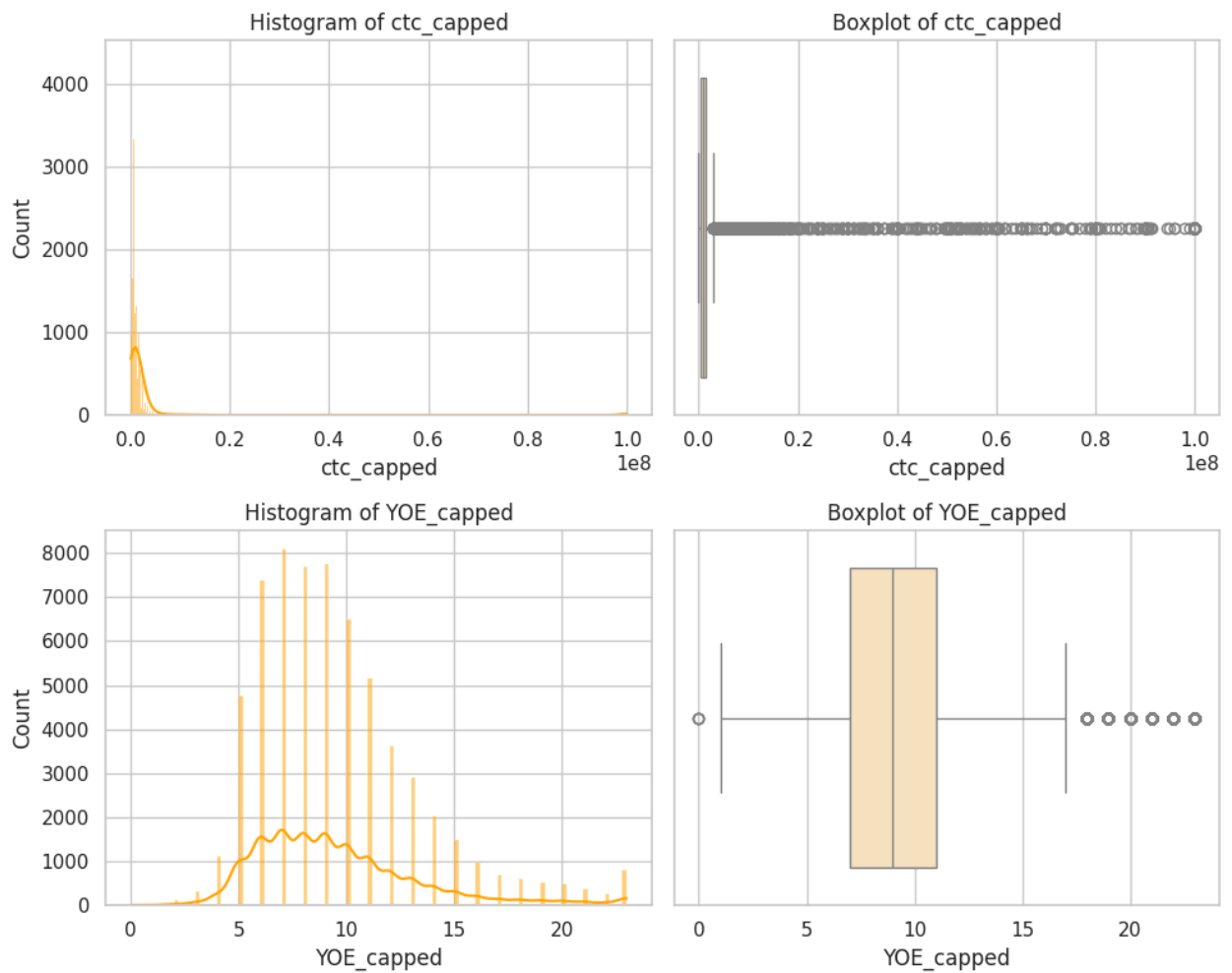
Numerical Feature Distribution

```
In [98]: plt.figure(figsize=(10, 8))

# Loop through each numerical column and plot histogram and boxplot
for i, col in enumerate(num_cols):
    # Histogram
    ax1 = plt.subplot(2, 2, 2*i + 1)
    sns.histplot(df3[col], kde=True, color='orange')
    plt.title(f'Histogram of {col}')

    # Boxplot
    ax2 = plt.subplot(2, 2, 2*i + 2)
    sns.boxplot(x=df3[col], color='moccasin')
    plt.title(f'Boxplot of {col}')

plt.tight_layout()
plt.show()
```



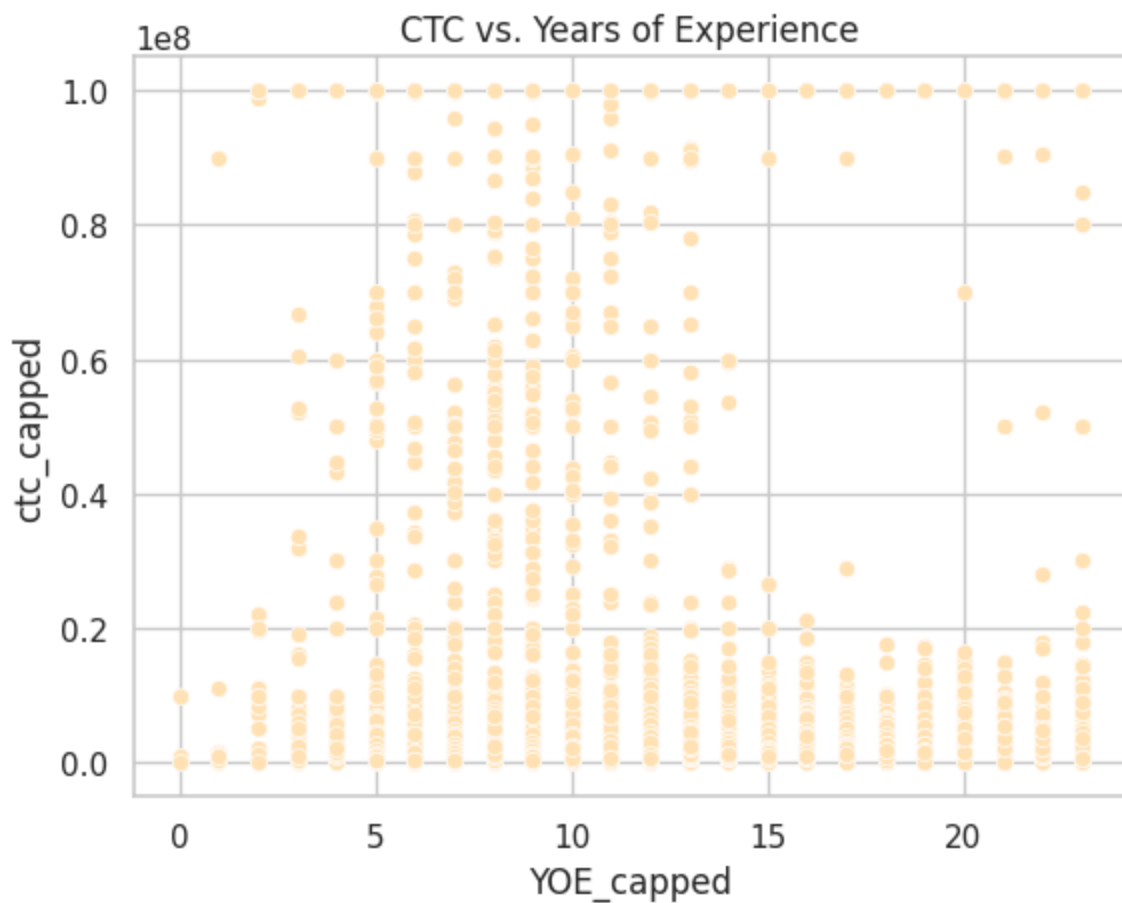
Distribution of CTC is right skewed

Distribution of YOE is almost normal with most of the YOE lying around 6-9 years

Bivariate Analysis

CTC vs Years of Experience

```
In [99]: sns.scatterplot(x='YOE_capped', y='ctc_capped', data=df3, color='moccasin')
plt.title('CTC vs. Years of Experience')
plt.show()
```



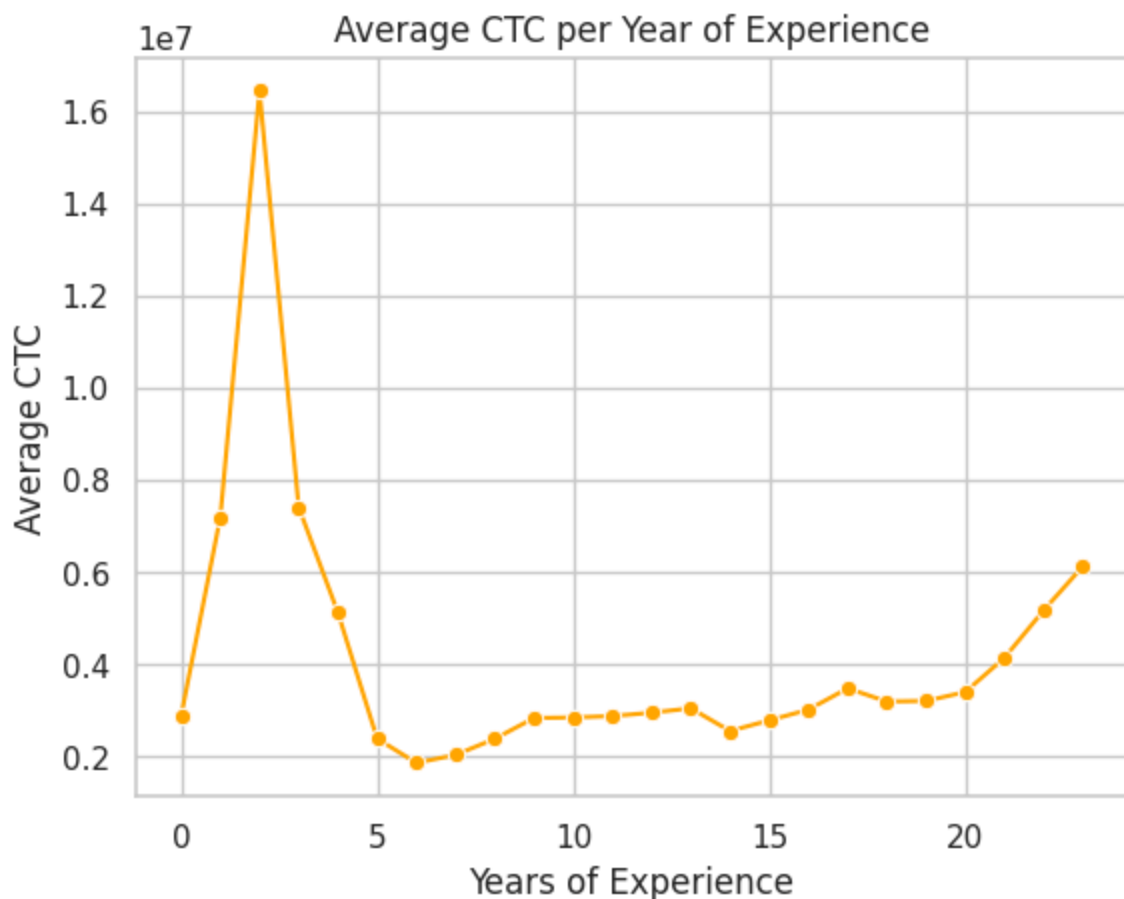
There is no linear relationship or any specific pattern between Years of Experience and CTC

Years of Experience vs Avg. CTC

```
In [100... # Calculate average CTC per year of experience
avg_ctc_per_yoe = df3.groupby('YOE_capped')['ctc_capped'].mean().reset_index()

# Line plot with markers
sns.lineplot(x='YOE_capped', y='ctc_capped', data=avg_ctc_per_yoe, marker='o',
plt.title('Average CTC per Year of Experience')
plt.xlabel('Years of Experience')
plt.ylabel('Average CTC')

plt.show()
```



Avg. CTC is decreasing from 1 to 5 years of Experience. There might be a slight decrease in CTC with increasing experience, possibly due to industry-specific factors or career shifts.

Q. Name job position that is commonly considered entry-level but has a few learners with unusually high CTCs in the dataset.

```
In [101]: # Define common entry-level job positions
entry_level_positions = ['Junior Engineer', 'Intern', 'Trainee', 'Associate',

# Filter for entry-level jobs
entry_level_df = df3.query('job_position in @entry_level_positions')

# Aggregate mean and max CTC per job_position
agg_stats = entry_level_df.groupby('job_position')['ctc_capped'].agg(['mean',

# Filter positions where max > 2 * mean (unusually high CTC)
unusually_high_ctcs = agg_stats[agg_stats['max'] > 2 * agg_stats['mean']]

# Reset index to get a clean DataFrame
unusually_high_ctcs = unusually_high_ctcs.reset_index()

print(unusually_high_ctcs)
```

	job_position	mean	max
0	Associate	1.326000e+06	2840000
1	Intern	8.877778e+05	2000000

Associate is one job position considered as entry level but maximum CTC going beyond set threshold

```
In [102... # -----
# 1. Average CTC by Job Position
# -----
def get_average_ctc_by_position(df):
    """
    Calculate and return the average CTC for each job position,
    sorted descending by average CTC.
    """
    avg_ctc = (
        df.groupby('job_position')['ctc_capped']
        .mean()
        .reset_index(name='average_ctc')
        .sort_values(by='average_ctc', ascending=False)
    )
    return avg_ctc
```

Q. What is the average CTC of learners across different job positions?

```
In [107... avg_ctc_by_position = get_average_ctc_by_position(df3)
print(avg_ctc_by_position)
```

	job_position	average_ctc
448	Telar	100000000.0
37	Assistant manager	100000000.0
205	Owner	100000000.0
253	Reseller	100000000.0
72	Business Man	100000000.0
..	...	...
400	Some data entry operator like some copys write...	10000.0
16	Any technical	10000.0
186	Matlab programmer	10000.0
385	Software Enginner	6200.0
442	Technical consultant	2000.0

[491 rows x 2 columns]

```
In [103... # -----
# 2. Identify Entry-Level Positions with High CTC Outliers
# -----
def get_entry_level_high_ctc_outliers(df, entry_level_positions):
    """
    Identify entry-level job positions where max CTC > 2 * mean CTC,
    indicating unusually high CTCs for some learners.
    """
    entry_df = df.query('job_position in @entry_level_positions')
```

```
stats = entry_df.groupby('job_position')['ctc_capped'].agg(['mean', 'max'])

outliers = stats[stats['max'] > 2 * stats['mean']].reset_index()
return outliers
```

How can we identify entry-level employees who have unusually high CTC

```
In [108... entry_level_positions = ['Junior Engineer', 'Intern', 'Trainee', 'Associate',
high_ctc_outliers = get_entry_level_high_ctc_outliers(df3, entry_level_positio
print(high_ctc_outliers)
```

	job_position	mean	max
0	Associate	1.326000e+06	2840000
1	Intern	8.877778e+05	2000000

```
In [104... # -----
# 3. List Companies with 'Data Scientist' Role
# -----
def companies_with_role(df, role):
    """
    Return a list of companies where the specified job role exists.
    """
    role_df = df[df['job_position'] == role]
    return role_df['company_hash'].unique()
```

ow can we identify companies that have a specific role, such as 'Data Scientist', in the dataset?

```
In [109... data_scientist_companies = companies_with_role(df3, 'Data Scientist')
print("Companies with 'Data Scientist' role:")
print(data_scientist_companies)
```

Companies with 'Data Scientist' role:

['ihvznuyx' 'tqxwoogz' 'vrsgzgd ucn rna' 'xzntrrtwn atoxsz vqtzv rna'
'ihvrwgb' 'vagmt' 'cxqnhov' 'mqxonrtwgzt v bvyxzaqv sqghu wgbuvzj'
'owvznv' 'qtvr nxbt oxszvro ntwyzgrgsj' 'vmogrhnavnv vzvrjnxwo'
'xzthqgzvx' 'mggpbjoygf' 'nvnv wgzohrnvwj otqcxwto' 'tuvjrvntq' 'bga'
'ehkkj rgsxd' 'fvrbvqn rvmo' 'vzvrjnxwo cxayjv' 'bvontqvwqa'
'bxwqgotbx wgqugqvnvgz' 'vcxog vx' 'myvoyjvb owyggr'
'zxoovz axsxnv xzaxv' 'tznxstzwt' 'fzo srgmvr otqcxwto'
'pvqkv ntwyzgrgsj' 'erxuoygut' 'ntvb wgbuhntqo' 'avnv otbvznxwo'
'ftrro evqsg' 'ptnovvr qtnvvr rru' 'eqtoytq'
'ovbohs qa xzonxnhnt xzaxv atryx' 'wtzntrgz xn ogrhnxgzo'
'qtrvxwt xzahonqto rxbxnta' 'onhatzn' 'ihvznuyx xzw'
'nyxpmqast ogenfvqt' 'sggsrt' 'wvustbxz' 'yjutqogzxd' 'xzouhq'
'otzotsqvo' 'hznxntaytvrny sqghu' 'vsvznxyv wgszxnxt ogrhnxgzo'
'ihjntwy' 'vbkvgz' 'avnbxnto xzaxv ucn rna' 'ongqtpxzs' 'vmxzmte'
'ftmnhzxd' 'gqvwr' 'zgn vuurxwvmt vwwghzn' 'xzongqxta'
'ygqkxgz xzegqbnxw otqcxwto ucn rna' 'sggafgqprvmo' 'lgyzogz wgnqgro'
'vrsg8' 'eqtrvzwtq' 'xzegojo' 'ugrxwmkvvqwg' 'sqvm' 'vzk'
'rvntzncxtf vzvrjnxwo' 'nguuq' 'xwxw mvzp' 'wxnx' 'vbtqxwvz tduqtoo'
'mxsavnbvnxwv ogrhnxgzo' 'vavo' 'amo mvzp' 'owqumgd xzaxv ucn rna'
'wgszxbvzn' 'nton wgbuvzj' 'nvnv ontr' 'vrronvnt ogrhnxgzo ucn rna'
'uvqpkvuwg' 'bgqtjtvyo' 'uhwyg ntwyzgrgsj xzegqbnxgz uqxcvnt rxbxnta'
'grv vzx ntwyzgrgsxto ucn rna' 'bxzamgvqa xzw' 'qhmxd' 'wgzwtznqxd'
'avnbvtnxwv ogrhnxgzo uqxcvnt rxbxnta' 'wqtaxfvnw' 'vngo ojznt'
'ovvqnyxv' 'gwnqg xzw' 'vznrtq' 'ytastrgg' 'attr' 'dxrjnxwv'
'bxyhu wgbbhxwvnxgz' 'gjj' 'ngpgutaxv' 'btanqgzxw' 'ovo'
'gznvqxg ntvywtq utzoxgz urvz' 'ohwtoobtnqxw' 'xzaxvz owyggr ge bxzto'
'exzkvq' 'ftmo mtnrt xn ogrhnxgzo ucn rna' 'zxtrotz' 'ntrtutqegqbvzwt'
'qtrvxwt xzegwgb rxbxnta' 'gbtsv ytvnywvqt' 'bxzanqtt'
'btqwtatmtzk qtvtvqwy vza atctrubtzn xzaxv' 'zgcqnxo' 'xzzgbxza'
'xqgz bghznvz' 'vau' 'vwwtznht' 'bvyxzaqv wgbcxv'
'onhatzn fxny urvwtbtzn geetq' 'axbtzoxgz zds' 'eqvhygetq hov'
'eqtoyatop' 'ntwymxn' 'nvnqvo avnv otqcxwto' 'uytzb utgurt'
'nvnv vzvrjnxwo xzoxsyno' 'ywr ntwyzgrgsxto'
'yxnvwyx ogrhnxgzo vbtqxwv rna mhoxztoo ogrhnxgz sqghu' 'st ytvnywvqt'
'jvoy ntwyzgrgsxto' 'tdr vzvrjnxwo' 'pgnvp rxet' 'attr ntwyzgrgsxto'
'mtwpbvz wghrntq' 'nyt a t oyvf sqghu' 'fxuqg rxbxnta' 'xwxwmxvzp rna'
'uvjnb' 'uqgcxot wgzohrnxs' 'nxstq vzvrjnxwo' 'utqtzsg' 'nvxjg rrw'
'ctqxkgz' 'srgmvr rgsxw ogenfvqt ntwyzgrgsxto' 'bxwqgogen' 'gzturho'
'rvqotz vza nghmqg xzegntwy' 'ktqg wgbuvzj utqegqbvzwt bvqptnxzs'
'hzxctqoxnj ge bvoovwyhotno vbytqon' 'hnguho xzoxsyno' 'ayvz vx'
'zvsvqqg' 'uqtaxwnxtjt' 'wvqttq rvhzwytq' 'wrr' 'uqgcvzv'
'pgzwtn qtvtvqwy' 'bvznvyz' 'mqxrrxg' 'pfxntwy' 'xb v onhatzn' 'aqvhu'
'oyxuoj' 'zgn jtn fgqpxzs vb onxrr onhajxs' 'ztfstz ogenfvqt'
'stbxz ogrhnxgzo' 'tqxwoogz srgmvr otqcxwto xzaxv ucn rna'
'ztrv vzvrjnxwo' 'ogwxtnt stztqvt srgmvr ogrhnxgz wtzntrq' 'hztburgjta'
'qtexznxc' 'ztnogr ntwyzgrgsxto xzw' 'nvnv wgzohrnvwj otqcxwto ucn rna'
'ihvznkxs' 'xzattawgb' 'ogwxn szqvt' 'ouxwt bgztj' 'sgctqbtzn ge xzaxv'
'atrgxnt' 'dwtavzwt' 'st' 'vuurxta bvntqvxro' 'eqvwnv vzvrjnxwo'
'stztqvr bxrr' 'ztcxngz ogentwy ucn rna' 'bvxr bvnt' 'dtgzonvwp'
'atbvnxw qtaaftqpo' 'zthatow' 'atwxxgz qtohqwto sqghu' 'bh oxsbv xzw'
'bzw rna' 'oxbtzo' 'urvav xzegntwy' 'srgmvrrgsxw' 'nqvwnxgz gz atbvza'
'ovbohs trtnqg btyvzxwo' 'nwo xzgcvnvgz rvmo' 'btwy bgwyv'
'whqr vzvrjnxwo' 'kjrgntwy' 'wqtaxncxajv' 'shvcho' 'zhqng']

'vwxw ntwyzgrgsxto ucn rna' 'bgqsvz onvzrtj' 'exg rvmo' 'ofxssj' 'ovu'
'tvonbvz wytbxwvr wgbuvzj' 'szvzxvx' 'xwhmt wox' 'ghnowvr'
'vwyvrv xn ogrhnxgzo ucn rna' 'ouqxzprq' 'yjhzavx vhngtctq xzaxv'
'ytdvfvqt ntwyzgrgsxto' 'bvyvqvoynqv pzgfrtast wgqugqvnvgz rxbxnta bpwr'
'whqtexn' 'bnw' 'otrrwqven srgmvr ogrhnxgzo ucn rna'
'ntrvctqst wgbbhzxwvnvgz o ucn rna' 'oux srgmvr' 'tj'
'xzegsvxz xzaxv uqxcvnt rxbxnta' 'oytrr' 'nwo qtotvqwy' 'xuogo'
'ojztwyqgz' 'yxsyqvaxho' 'vqxosrgmvr' 'exznqhon ogrhnxgzo ucn rna'
'onqxut' 'shxatyghot' 'wqgfz tihxubtzn wgqugqvnvgz'
'svqnztq xzaxv qtotvqwy vza vacxogqj otqcxwto' 'cxov' 'jhrh mxpt'
'nvcvzn' 'ofxoo qt' 'hmo' 'pcvznhbvx' 'xmb' 'vvqvzjv wzohrnvwzj'
'bgnxefgqpo' 'wyxonvno rvmo ucn rna' 'utqxng wjmtq bvzvstbtzn ucn rna'
'ntwyzgogen wgu' 'rvqotz nghmqg' 'wrgctqntd'
'xzexzxnq qtotvqwy ucn rna' 'exwg' 'nojointbo xzntqzvnvgzvr sbmy'
'atryxctqj' 'stzuwn' 'xzntqz vn gjg qggb' 'xaej' 'qgvat rvmo'
'xzaxv xzegrxzt' 'fyxntyvn lq' 'uqgatcvzo ntwyzgrgsj' 'vjhqctav ptzaqv'
'wvuxrrvj ntwyzgrgsxto' 'nwo axsnvr tzntquqxot'
'wqxbogz tztqsj tdutqno uqxcvnt rxbxnta'
'nyvuvq xzonxnhnt ge tzsxztqxs vza ntwyzgrgsj' 'wqtaxn ohxoot'
'bvi ogenfvqt' 'mdm axsnvr' 'ptzgbt' 'yvxphlvb' 'exo'
'tdutqxgntwy uqxcvnt rxbxnta' 'tdvwnouvwt' 'ytrrgoxcx ogenfvqt rvmo'
'vwyvrv ogrhnxgzo' 'sqtnv ogenfvqt rvmgqvnvgj' 'vxtzntquqxot'
'sqvvr qtotvqwy' 'gnytt' 'ytdvsgz vm' 'oqxstzwxv xn ogrhnxgzo' 'zgzt'
'ztnvuu' 'axkxcxkx' 'ahugzn ohonvxzvmrt ogrhnxgzo' 'tlvav'
'fxotrtva xn ogrhnxgzo ucn rna' 'ogwxvrwguo' 'mrwvponqvf'
'mqvzt tzntquqxoto' 'wgzwtqngvx' 'wqgfavzvrjnx' 'pzgfrtast rtzo'
'otre tburgjta' 'ntwy bvyxzaqv' 'vdxqv xzstzxgho xzoxsyno'
'ihvaqvnw xzoxsyno ucn rna' 'srgmvzn' 'znfxon' 'imhqon'
'myvqvnjv lvzvnv uvqnj' 'atrgxnnt xzaxv' 'wxvzo vzvrjnxwo' 'nnv trdox'
'fot' 'xnw xzegntwy' 'otre tburgjta uqgetooxgzvr' 'avnv yxct rvmo' 'ko'
'zgn fgqpxzs' 'nvrtznxwv ogenfvqt' 'dwrwpo' 'nyt bnv wgbuvzj'
'stztqvr trtnw' 'eqvwnvr' 'pxoyn axsnvr rtzaxzs'
'ytftrnn uvwpvqa tzntquqxot' 'vbzo xzaxv' 'lubgqsvz wyvot wg'
'tvngz wgqugqvnvgz' 'vhqgzugq ogrhnxgzo rna' 'hsbv' 'onjrhbxv'
'voygp rtjrvza' 'adw ntwyzgrgsj' 'bxzaonxv ogenfvqt rvmo' 'hmtq'
'owqj vzvrjnxwo' 'qta yvn' 'xzatstzt twwbv'
'nnv wzohrnxs tzsxztq rxbxnta' 'vwvrcxg ntwyzgrgsxto' 'twgb tduqtoo'
'wrghaotp' 'tbxao' 'two xzw' 'rgsxntwy' 'vmkggm' 'owxtzvunw jointbo'
'wgzctqovzn rrv' 'nyt lgyzo ygupxzo hzxctqoxnj' 'nfg qgvao' 'ftmytru'
'uxcgnwyvzx ogrhnxgzo' 'xzaxvuqgutqnjwgb' 'vcvay vvatbj xzntq wgrtst'
'erjndn' 'vuqv rvmo' 'onvqn hu' 'zxjg ogrhnxgz' 'oxsnhurt' 'lhonagw'
'otvqo ygraxzso wgqugqvnvgz' 'utnqgmvo' 'mgtxzs' 'zeg tast xzaxv rna'
'tdr' 'erjzvcv ntwyzgrgsxto' 'x vb v eqtoytq' 'hnguxv srgmvr'
'nyxqa ovwt vhn' 'vxnjv mxqr exzvzt rna' 'yvjo mhoxztoo ogrhnxgzo'
'axqtnx' 'cbfvqt' 'ntwyzgogen srgmvr otqcxwto'
'yvuuxton bxzao ntwyzgrgsxto' 'wvzfxrr ntwyzgrgsxto'
'vrrtz wvqttq xzonxnhnt' 'zvz' 'pvzx ogrhnxgzo'
'xzntqwgznxztznvr tdwyvzst' 'bobtd' 'ihxdj' 'oguqv ontqxv' 'wsgugqn'
'r vza ntwyzgrgsj otqcxwto' 'trtcgz' 'bzp xzgcvnvgzo' 'gunhb' 'bwpxzotj'
'zeggmllwno xzw' 'xzbgm' 's2' 'vzvrjon' 'bvnt rvmo'
'atrgxnnt xzaxv geexwto ge nyt ho' 'pgnvp' 'gunhb srgmvr ogrhnxgzo'
'xmb xzaxv uqxcvnt rxbxnta' 'ntwyxowyt hzxctqoxnn mtqrxz'
'ktonxgn ntwyzgrgsxto' 'lmb sqghu uhmrxw rna' 'wyvqntq wgbbhzxwvnvgz'
'ovbohxs qa xzonxnhnt xzaxv mvzsvrgqt' 'ihvznvbxv ogrhnxgzo' 'ozvuqvuxa'

'dtqgd' 'oxsbgxa' 'bvdxb xzntsqvnta'
'zgbhqv qtotvqwy exzvzwxvr ntwyzgrgsxto' 'otrxot qgwpzx ogenfvqt'
'exctqq' 'yhaarj xzw' 'xeb tzsxzttxzs' 'uvzvogzxw' 'otzotxyhm xzw'
'stnhu egq wyvzst otqcxwto ucn rna' 'ugntyvrvm' 'bjznqv'
'nyt xzatutzatzn' 'mxno uxrvzx yjatqvmva wvbuho' 'aghmznzhn' 'tvngz wgqu'
'tdutqxgz' 'zv' 'srvdgobxnyprxzt uyvqbvwthnxwvro' 'hzxrtctq'
'nxcg wgqugqvnxyz' 'buyvxo' 'nan' 'ptzoyg ntwyzgrgsxto'
'lhzsrtt svbto xzaxv ucn rxbxnta' 'yaew mvzp'
'dug rgsxoxw fgqrafxt xzaxv ucnrna' 'mvjtq' 'fxuqg axsxnv'
'vpvxpt ntwyzgrgsxto' 'uqgojo' 'owyrhbmqtstq' 'ufw oaw' 'mgowy' 'vuurxen'
'otbxtzo' 'eqtrvzwt atctrqutq' 'wrtvqnqxuwb' 'rtdxoztdxo'
'xzexztgz ntwyzgrgsxto' 'avoovhrn ojonbto' 'xzaxd' 'oyguwrhto'
'vpvbx ntwyzgrgsxto' 'wxowg' 'ygztjfttr ntwyzgrgsj rvm' 'xztrtvuo'
'dug rgsxoxw xzw' 'orxzs btaxv' 'ovuxtznznqg' 'i3 ntwyzgrgsxto'
'hqmvmxj bhrnxogr uqxcvnt rxbxnta bjaxaxz' 'xzzgcg ogrhnxgzo'
'vwo ogrhnxgzo' 'stzj rvm' 'xzoxatcxtf ntwyzgrgsxto' 'wvqttb'
'tvoj qtfvqak' 'rgsbtzx wxnxd grxzt' 'sqvjagz hp'
'vxs qtotvqwy vzvrjnxwo ucnrna' 'pvuxnvrntwy' 'xzegonqtnwy' 'gunjb'
'rtzaxzspvqn' 'wv ntwyzgrgsxto' 'ovuxtzn' 'xzoget' 'u3 wzohrnxs'
'obtwgqztq' 'mtz shqgz hzxctqxonj xoqvr' 'zwq wgqugqvnxyz' 'atrnd'
'jhbuxzsg' 'ihxrvx' 'jto mvzp' 'qvpovz' 'nom rrgjao hp'
'tzahqvzwt xzntqzvnxyzvr sqghu' 'bxwqgogen wgqugqvnxyz'
'jnqqt ogenfvqt ogrhnxgzo ucn rna' 'utqcvwxg xzw' 'utqoxontzn ojonbto'
'yomw ogenfvqt atctrqubtzn xzaxv' 'lav ogenfvqt' 'xyhm xn ogrhnxgzo'
'ctqv otwhqxnj' 'lhon axvr rxbxnta' 'srgmvrast xzntrrxstzwt ge nyxzso'
'tastctqct' 'nvncxwo vzvrjnxwo' 'btqxrjnxwo xzw' 'otvqo xn bvzvstbtzn'
'pzgfaxo avnv owxtzwt' 'vuuzgbxw ojonbto' 'stwpgrjon'
'avnvq wvzwtq stztnxwo rna' 'pvznvq wzohrnxs' 'zxwt vwnxbxkt'
'ogzvnv ogenfvqt' 'hxxbgzx srgmvr mhoxztoo otqcxwto'
'oyvagfevd ntwyzgrgsxto ucn rna' 'qtwgbbtztatq rvm' 'srgbtd'
'otzcxgz sbmy' 'ou srgmvr' 'ehlxnoh wzohrnxs xzaxv ucn rna'
'hzrxbxn xgn ucn rxbxnta'
'agwovuu wzohrn v outwxvrxon agwngq xz 30 bxzhnto'
'hxxctqoxnj ge wgrgqvag vn mghratq' 'gztvooxon wzohbtq ogrhnxgzo'
'qvzaonva' 'xznhn' 'kgyg wgqugqvnxyz' 'vwnxej avnv rvm' 'wghnhqtvx'
'lxg' 'sgmxmg' 'qohnqv vzvrjnxwo vza wzohrnxs ucn rna' 'exotqc'
'bgmxrt uqtbxtq rtvsht bur' 'xzctznhb ntwyzgrgsxto' 'ugqzyhm'
'zgqny avpgnv onvnt hxxctqoxnj' 'st qtztfvmt tztqsj'
'zhnvzdn ntwyzgrgsxto ucn rna' 'whbbxzo ntwyzgrgsxto xzaxv rna'
'srgmvrrgsxw xzw' 'xztrotz xzaxv uqxcvnt rxbxnta' 'bwpxzotj vza wgbuvzj'
'veexzt vzvrjnxwo' 'stvp bxzao' 'abx exzvzwt ucn rna'
'mtrwvz otqcxwto sqghu' 'egqa bgngq wgbuvzj' 'nvqvo avnv' 'lvmgzs'
'dr ytvry' 'shqhwhr ogrhnxgzo' 'ogwxvruqvwyvq' 'zgpv' 'tcw ctznhtqto'
'cxontgz ntwyzwxvr vza otqcxwto wtznqt' 'xbuqtooxwg mhoxztoo ogrhnxgzo'
'votv mqgfz mgctqx' 'egwtugxzn' 'mxqrvogen' 'xxn yjatqvmva' 'bjvrrj'
'vba' 'btnqxuuxzs' 'uvqvaxsb bnhxnj' 'uqgszgovx'
'rtdxzzgcg ntwyzgrgsxto ucn rna' 'twg obvqn fvrr wgbuvzj'
'mqgvaqxast exzvzwxvr ogrhnxgzo xzaxv uqxcvnt rxbxnta' 'mvzp ge vbtqxwv'
'bojo ntwyzgrgsxto rrw' 'bvyxzaqv bvyxzaqv' 'hxxnta ytvry sqghu gunhb'
'tbgnxd' 'rtvqxxzs cxv grxzt wghqoto' 'cxaggrj' 'fytrj'
'xmb xzaxv uqxcvnt rna' 'bxi axsxnv' 'avnv wgqt ojonbto'
'3ix rvm ucn rna' 'wgaxeja xzw' 'vcvrx ogrhnxgzo' 'ogrcdq ogrhnxgz'
'ktt tzntqnvxzbzn tzntquqxoto' 'yomw' 'xptv'
'sqttz ugxn srgmvr otqcxwto' 'uvsg vzvrjnxwo xzaxv'

'gnox gmltnw ntwyzgrgsj ogrhnxgzo xzw' 'wgqt wgbutnt'
'zgpvx wgqugqvnvgz' 'exmqg tantwy ucn rna' 'zghe xzegovontbo'
'ogrvq xzahonqxta xzaxv rna' 'zvcx' 'ztnvq' 'bwbvqcxz' 'ohmtd' 'iuxvx'
'lxg urvnegqbo' 'qgjvr wgzzvhsyn mgvn wrhm' 'fvjwggrr vmo'
'mxgotzot ntwyzgrgsxta ucn rna' 'ovrtoegqwt' 'kjuu trtnwnqxw'
'xqxo ogenfvqt xzw' 'wgeghzatq vn fvoypvqn' 'yvunxp' 'vactzgx'
'tcvrhtotqct' 'yxrn' 'xmo ogenfvqt' 'ahzzzhybmj'
'xzegqbnvgz owxtzwt xzonxnhnt' 'uqttbxztzwt'
'gunxbvr onqvntsd sqghu xzw' 'wtrtmvr ntwyzgrgsxta' 'nyqtahu' 'bwvett'
'uyxrvatruxv 76tqo' 'aqtvb11' 'xuqxbtavnv' 'uygztut'
'gcxa wvuxnvr rrw nyhzipfgd' 'zhbmtq nytgqj' 'onvqnhu' 'mrwvpggwp'
'hzxctqoxnj ge wgrgqvag mghratq' 'cxqnhovugrvqxo'
'ivnvq wgbuhnxs qtotvqwy xzonxnhnt' 'vhngatop'
'zgpvx ogrhnxgzo vza ztnfgqpo' 'avngxz' 'rtnoctznhqt'
'uxzzvwr axsnvr vzvrjnxwo ucn rna' 'gwnguvqn' 'uhzt'
'ojqvwhot hzxctqoxnj' 'mvqwrjvjo wgqugqvnt mvzpxzs' 'vhxmrt xzw'
'bowx xzw' 'rn ntwyzgrgsj otqcxwto rxbxnta' 'znn avnv otqcxwto'
'vnytzvytrny' 'st axsnvr' 'ktzaqxct' 'mtrgzswg' 'qxctqmta ntwyzgrgsj'
'avxrjyhn' 'kvuq btaxv vmo' 'tvojqtqvqak' 'sgrabvz ovwyo' 'oyxuuq'
'yhmd' 'ctqot' 'xznvz ntwyzgrgsxta' 'yomw trtnwnqgzxw avnv uqgwtooxzs'
'nygbogz axsnvr' 'souvz ntwyzgrgsxta xzw' 'qvzx xzahonqxta'
'uqhatznvr srgmvr otqcxwto' 'ztn srgmvr' 'mn' 'wgbowgqt xzw' 'uvjuvr'
'uqvbvn ntwyzgrgsxta uqxcvnt rxbxnta'
'ihvznxuyx vzvrjnxwo ogrhnxgzo ucn rna' 'yqvbnz wgzztwna otqcxwto'
'xbuvwn vzvrjnxwo' 'myvqn exzvzwxvr xzwrhoxgz rxbxnta' 'ehlxnoh srgmvr'
'xqwnw' 'sqvbtztq' 'vhngbnxw avnv uqgwtooxzs' 'ihvzntrv'
'dgqvxzn ogrhnxgz ucn rna' 'lhdnobvqn bvzavnt ucn rna' 'a whmt vzvrjnxwo'
'wgnxcnx' 'qgrroqgjw' 'aqag mvzsvrgqt' 'pvqj vzvrjnxwo' 'ayxgbxwo'
'ihvrwgb xzaxv ucn rna' 'onvzavqa wyvqntqta mvzp' 'ctzbogr'
'vxqex vcnvnxgz ucn rna' 'mvzp ge vbtqxwv wgqugqvnvgz' 'xixv'
'zgcqvnxo ytrnywvqt' 'ogzvnv ogenfvqt rxbxnta' 'avqvoyvf'
'rvqotz vza nghmqg xzegtwy vshbtznxi'
'stztqvr trtnwnqxw xzntqzvnvgzvr wgqugqvnvgz' 'ctznqxn xzw' 'xyo bvqpxn'
'onhatzn mntwy exzvr jtvq' 'wgzohbtqnvwp' 'vunho avnv vmo'
'owqj vzvrjnxwo xzaxv ucn rna' 'wvbuhotvx wgzogqnxhb'
'nvkvu ucn rxbxnta' 'gutx xzoxsyno rru' 'otihqtnp'
'mvjoygqt xzntrrxstzwt ogrhnxgzo' 'ojbuygz vx'
'rtsvng ytrny ntwyzgrgsxta' 'soz nc svbto' 'sjqho vx' 'wgegqst'
'atrgxnt ho xzaxv' 'zvqvjvzv tahwnvgz ogwtnj' 'vzgnytq wgbuvzj'
'ztnrxzp ogenfvqt' 'zhwrtho ogenfvqt' 'atbvza bvnqxd'
'obvqnyhzy unt rna' 'ctavzn' 'oqxlvz ntwyzgrgsxta' 'yvw vza wqven'
'rvqotz nghmqg xzegtwy rna' 'dtmxv sqghu xzntqzvnvgzvr' 'nyt avnv ntvb'
'lhxzutq ztnfgqpo' 'nwo xgz' 'nqvzngq' 'bvnyfgqpo yjatqvmva'
'exxoyxzs owyggrr' 'mrvxkt' 'wjxtzn xzoxsyno' 'btqxrjnxwo' 'zvqqvqt'
'utzzjmvt ogrhnxgz ntwyzgrgsj' 'ctqoxct' 'urhbxzw' 'svw qa wtzntq'
'zthonvq xzw' 'zxxn ntwyzgrgsxta rxbxnta' 'ovbohzs trtnwnqgzxwo'
'ouggpexoy xxxgcnvgzo' 'mzj btrrgz ntwyzgrgsj xzaxv'
'ovbohzs qtotvqwy xzonxnhnt zgxav' 'exzavmxrnj owxtzwt' '247vx'
'evwnotn' 'tznqtqtzthqoyxu wtrr zxn qghqptrv' 'cxvvgzztn' 'fvrbvqn'
'nxajihvzn' 'pubs' 'stu fgqafxat' 'vnntznxct vx' 'wgzwxqgho'
'cvwg mxzvqj otbvznxwo rru' 'khzqgge ntwy uqxcvnt rxbxnta' 'erxupvqn'
'qtrxvzwt axs urv uqg otq rna' 'fyvnexdwgb' 'otqcxwtzgf' 'havvzwb'
'zcxav sqvuxwo' 'rtsvng ytrny ntwyzgrgsj' 'atzvct xzaxv ucn rna'
'qvl ltftrrtqo rhwpzgf uqxcvnt rxbxnta' 'ygztonmtt'

```

'eqvzprxz ntburtngz xzctonbtzno' 'hufgqpwgb' 'wxurv rxbxnta' 'vxqntr'
'ovbohzs atryx' 'ihvznxuyx' 'exztrvmo ucn rna' 'onvoyexz'
'cxntgo wvuxnvr bvqptn otqcxwto' 'wtzntqavnn' 'xmb xz' 'vjvnn'
'vcgz ntwyzgrgsxto' 'lhbu360' 'owxtzvunxw ojointbo uqxcvnt rxbxnta'
'ftontqz hzxgz' 'vzzgnvnxgz vzvrjnxwv' 'cxqnhov wgzohrnxs otqcxwto'
'pgpg ztnfgqpo' 'hon srgmvr' 'oxtbztzo ytvnyxztqo' 'onjrvmo'
'os vzvrjnxwo' 'vxs' 'zxcvnnv ojointbo' 'tuvb ojointbo' 'svxzoxsyn'
'zgzt eqtoytq' 'vngo xzaxv ucn rna' 'vm xzmtc'
'wjntr onvnxonxwvr ogenfvqt otqcxwto ucn rna' 'pvmmvst xzw' 'wxbm mvzp'
'mv wgnzxzhbmvpz ge vbtqxwv' 'cxoxmrt vruyv' 'ovqv ntdnxrto rna'
'wvntxzv ntwyzgrgsxto' 'eqvzprxz ntburtngzxzzgc' 'tzihtqg srgmvr'
'vunvqv' 'trvfgbvz' 'xpvqho' 'pzgfrtast rtzo ucn rna'
'sqvbguygzt nqvzoegqbxzs vsqxwhrnht' 'woo wgqugqvnt'
'gwtvz ftm ntwyzgrgsxto' 'kxzzgc bvzvtbtzn wgzohrnxs' 'ovhax vqvbwg'
'ztnobvqnk' 'oxqgxzrvmo' 'wgzextznxvr'
'vowtzn ytvny vza ftrrtzo ogrhnxgz ucn rna' 'ktgnvu'
'btqwtato mtzk qtotvwy atctrubtzn xzaxv' 'ntwycvznvst ojointbo'
'adw ntwyzgrgsxto' 'q ojointbo xzntqzvnxgzvr' 'xdxsgwgb' 'wxbm bvrvjoxv'
'trtbtzn xzegbnxwo' 'oxburxevvx' 'nvnv wgbhbxwvnxgz rna'
'dvcxtzn axsnvr ugftqta mj ntrho xzntqzvnxgzvr' 'bggeqgs rvmo'
'obvvoy tzntqnvxbtzn' 'rhwxatho' 'vqjzs' 'xzoxatcxtf ntwyzgrgsxto xzw'
'rgjrnj qtfvqak' 'hznqgaatz rvmo'
'vghmv v yftrtn uvwpvqa tzntquqxot wgbuvzj' 'vsgav'
'wktznqxd ngfvqao cxoxgz ntwyzgrgsxto ucn rna' 'prghaavnn rvmo'
'lbl exzvzwxvr' 'sggz onhaxgo' 'vuurt'
'ohzqxot ongw mqppxzs uqxcvnt rxbxnta' 'sogw' 'xznt wqgugqvnxgz'
'ihvtqg' 'qht sxrn sqghut' 'btaxmgd' 'vznqxt'
'yvoytaxz ntwyzgrgsxto ucn rna']

```

In [105...

```

# -----
# 4. Compare Average CTC of 'Data Scientist' vs Other Roles in a Company
# -----
def compare_data_scientist_ctc(df, company_hash):
    """
    For a given company, compare average CTC of 'Data Scientist' role
    against other roles.
    """
    company_df = df[df['company_hash'] == company_hash]

    ds_ctc = company_df[company_df['job_position'] == 'Data Scientist']['ctc_c
    other_ctc = company_df[company_df['job_position'] != 'Data Scientist']['ct

    print(f"Average CTC for Data Scientist in company {company_hash}: {ds_ctc}")
    print(f"Average CTC for other roles in company {company_hash}: {other_ctc}")

    if pd.isna(ds_ctc) or pd.isna(other_ctc):
        print("Data Scientist role or other roles not found in the company.")
    else:
        if ds_ctc > other_ctc:
            print("Data Scientists have a higher average CTC compared to other")
        elif ds_ctc < other_ctc:
            print("Data Scientists have a lower average CTC compared to other")
        else:
            print("Data Scientists have the same average CTC as other roles.")

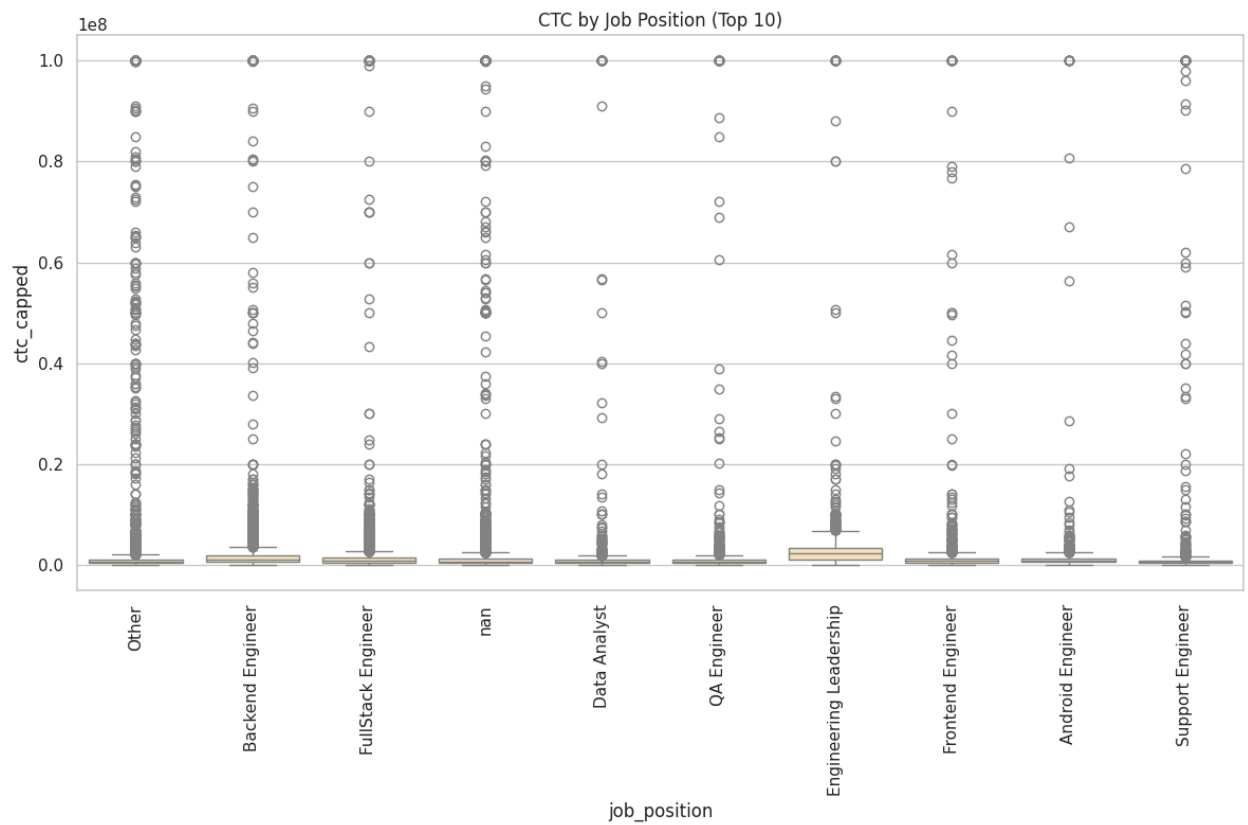
```

```
In [110... compare_data_scientist_ctc(df3, 'xznhxn')
```

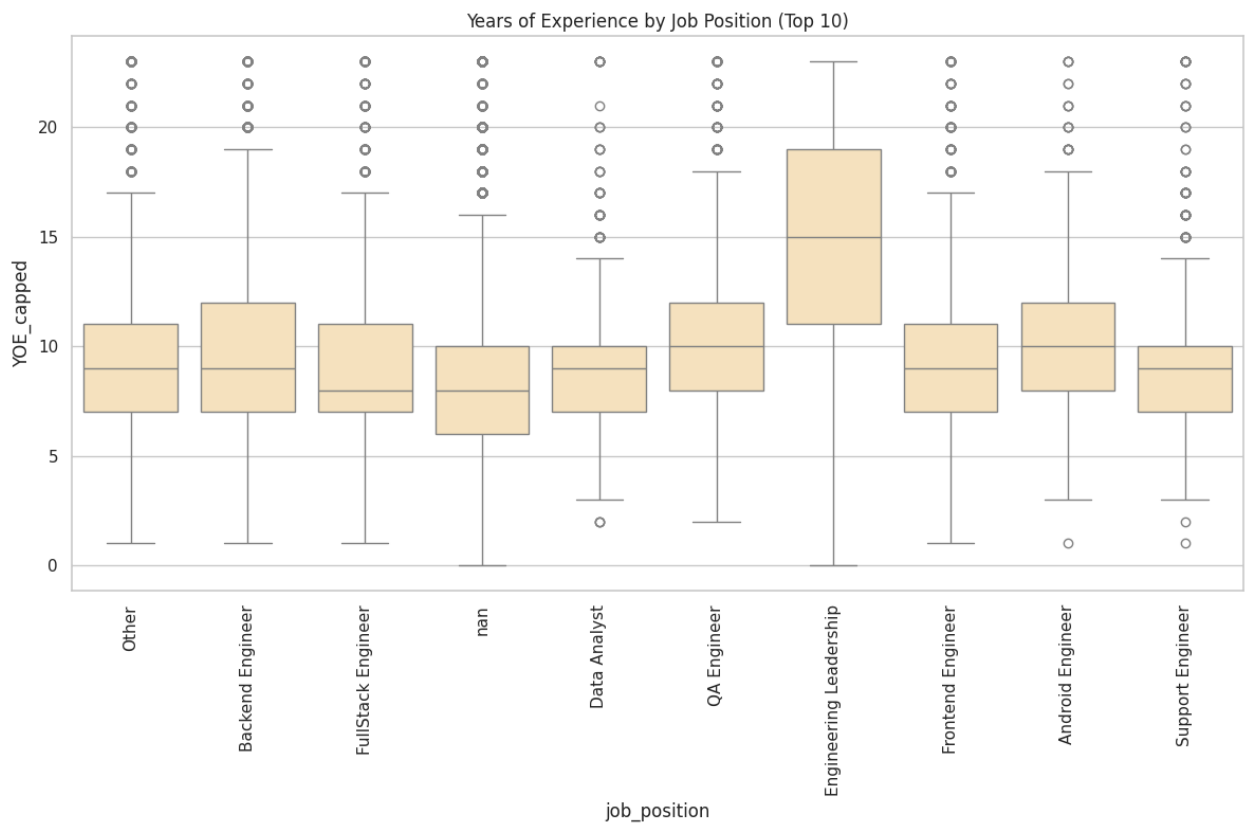
Average CTC for Data Scientist in company xznhxn: 3500000.0
Average CTC for other roles in company xznhxn: 2649483.8235294116
Data Scientists have a higher average CTC compared to other roles.

```
In [106... # -----  
# 5. Visualize CTC and Years of Experience by Job Position  
# -----  
def plot_top_positions_boxplots(df, col, title, top_n=10, color='moccasin'):  
    """  
    Plot boxplots of specified column grouped by top N job positions by frequency  
  
    Parameters:  
    - df: DataFrame  
    - col: column to plot on y-axis ('ctc_capped' or 'YOE_capped')  
    - title: plot title  
    - top_n: number of top job positions to include  
    - color: boxplot color  
    """  
    top_positions = df['job_position'].value_counts().nlargest(top_n).index  
    df_top = df[df['job_position'].isin(top_positions)]  
  
    plt.figure(figsize=(12, 8))  
    sns.boxplot(x='job_position', y=col, data=df_top, color=color)  
    plt.xticks(rotation=90)  
    plt.title(title)  
    plt.tight_layout()  
    plt.show()
```

```
In [111... # Plot CTC distribution by top 10 job positions  
plot_top_positions_boxplots(df3, 'ctc_capped', 'CTC by Job Position (Top 10)')
```

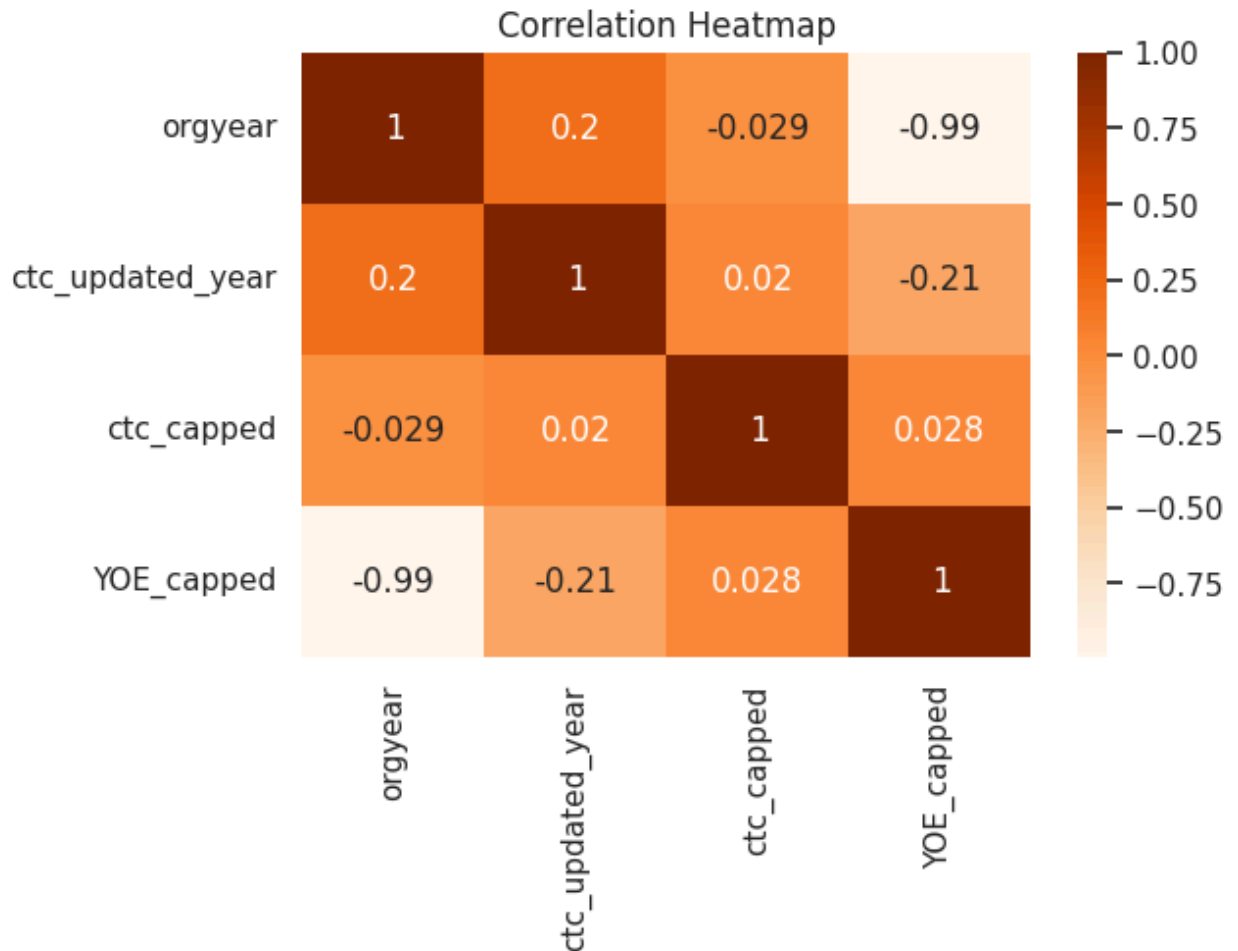


In [112... `# Plot Years of Experience distribution by top 10 job positions`
`plot_top_positions_boxplots(df3, 'YOE_capped', 'Years of Experience by Job Pos`



Correlation Heatmap

```
In [113... plt.figure(figsize=(6, 4))
sns.heatmap(df3[['orgyear', 'ctc_updated_year', 'ctc_capped', 'YOE_capped']].c
plt.title('Correlation Heatmap')
plt.show()
```



orgyear and ctc_updated_year shown weak positive correlation

Years of Experience and orgyear show strong negative correlation

Years of Experience and CTC show weak positive correaltion

```
In [114... df2.describe(include='object')
```

	company_hash	email_hash	job_p
count	63529	63529	
unique	17834	63529	
top	nvnv wgzohrnvzwj otqcxwto	2c55c5ffb21d0394312ec7acf6b7cd007329e2679a5c82...	
freq	3124		1

```
In [115... df2.describe()
```

	orgyear	ctc	ctc_updated_year	YOE	ctc_capped	YOE_
count	63529.0	6.352900e+04	63529.000000	63529.0	6.352900e+04	
mean	2015.46626	3.284883e+06	2019.787137	9.53374	2.694774e+06	9
std	3.97669	1.791946e+07	1.314355	3.97669	1.177649e+07	3
min	1970.0	2.400000e+01	2015.000000	0.0	2.400000e+01	
25%	2014.0	5.000000e+05	2019.000000	7.0	5.000000e+05	
50%	2016.0	8.000000e+05	2020.000000	9.0	8.000000e+05	
75%	2018.0	1.500000e+06	2021.000000	11.0	1.500000e+06	
max	2025.0	1.000150e+09	2021.000000	55.0	1.000000e+08	

Data Processing for Unsupervised Learning

Encoding

Scaling

Feature Engineering

Log Transformation

```
In [116... df4=df3.drop(['Designation_Flag', 'Class_Flag', 'Tier_Flag'], axis=1)
```

```
In [117... dfcopy=df.copy()
```

```
In [118... # Calculate frequency of email_hash in dfcopy
email_hash_freq = dfcopy['email_hash'].value_counts().reset_index()
email_hash_freq.columns = ['email_hash', 'no_of_ctc_update']

# Merge with df4 on email_hash
df4_merged = pd.merge(df4, email_hash_freq, on='email_hash', how='left')
```

Created a new feature 'no_of_ctc_update' signifying the number of times CTC got updated of a learner which is derived from frequency of email_hash in the dataset

```
In [119... df4_merged=df4_merged.drop(['email_hash', 'orgyear', 'ctc_updated_year'], axis=1)
```

Removing following columns:

email_hash: It is unique for each row and do not provide useful information for clustering

orgyear: We have got Years of Experience derived from this feature which is more relevant then just the year of joining for clustering algorithm

ctc_updated_year: We have derived a feature no. of ctc update signifying number of times ctc got updated of a learner which is more relevant to clustering algorithm than mere year as a timeline or int

```
In [120... df4_merged.head()
```

```
Out[120... 
```

	company_hash	job_position	ctc_capped	YOE_capped	no_of_ctc_update
0	atrgxnnt xzaxv	Other	1100000	9	1
1	ojzwnvwnxw vx	Backend Engineer	2000000	10	1
2	ngpgutaxv	Backend Engineer	700000	8	1
3	lubgqsvz wyvot wg	FullStack Engineer	1500000	7	1
4	vwwtznht ntwyzgrgsj	Backend Engineer	400000	6	1

```
In [121... df5=df4_merged.copy()
```

Encoding non-numerical columns

Frequency Encoding replaces each categorical value with its frequency in the dataset. A good compromise between simplicity and capturing categorical variable importance.

```
In [122... # Frequency encoding for company_hash
company_hash_freq = df4_merged['company_hash'].value_counts().to_dict()
df4_merged['company_hash_encoded'] = df4_merged['company_hash'].map(company_hash_freq)

# Frequency encoding for job_position
job_position_freq = df4_merged['job_position'].value_counts().to_dict()
df4_merged['job_position_encoded'] = df4_merged['job_position'].map(job_position_freq)
```

```
df4_merged=df4_merged.drop(['company_hash', 'job_position'], axis=1)
```

```
In [123...] df4_merged.head()
```

```
Out[123...]   company_hash  job_position  ctc_capped  YOE_capped  no_of_ctc_update  com
```

0	atrgxnnt xzaxv	Other	1100000	9	1	
1	ojzwnvwnxw vx	Backend Engineer	2000000	10	1	
2	ngpgutaxv	Backend Engineer	700000	8	1	
3	lubgqsvz wyvot wg	FullStack Engineer	1500000	7	1	
4	vwwtznhqt ntwyzgrgsj	Backend Engineer	400000	6	1	

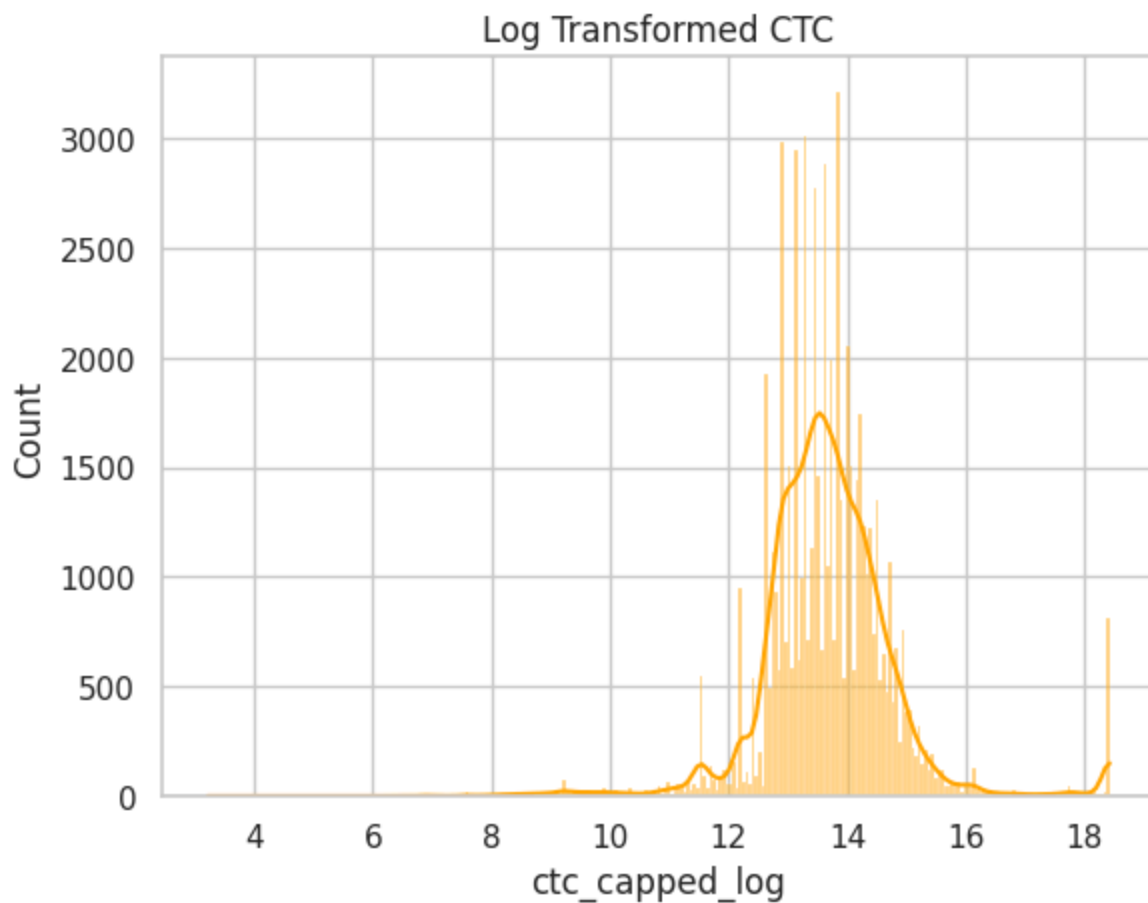
Log Transformation

Applying Log Transformation on ctc_capped column since it is right skewed. Since skewness can affect performance of clustering algorithms

```
In [124...] df4_merged['ctc_capped_log'] = np.log1p(df4_merged['ctc_capped'])
```

```
In [125...] df4_merged=df4_merged.drop(['ctc_capped'], axis=1)
```

```
In [126...] sns.histplot(df4_merged['ctc_capped_log'], kde=True,color='orange')  
plt.title('Log Transformed CTC')  
plt.show()
```



After applying log transformation, the feature shows normal distribution

```
In [127...] df4_merged.head()
```

```
Out[127...]
   company_hash  job_position  YOE_capped  no_of_ctc_update  company_hash_e
0  atrgxnt xzaxv      Other           9                1
1  ojzwnvwnxw vx      Backend Engineer          10                1
2    ngpgutaxv      Backend Engineer           8                1
3  lubgqsvz wyvot      FullStack Engineer           7                1
4  vwwtznhqt      Backend Engineer           6                1
   ntwyzgrgsj
```

Standard Scaling

```
In [128...] # Initialize the StandardScaler
scaler = StandardScaler()
```

```
# Fit and transform the data
scaled_features = scaler.fit_transform(df4_merged[['YOE_capped', 'no_of_ctc_up

# Convert the scaled features back to a DataFrame
df_scaled = pd.DataFrame(scaled_features, columns=['YOE_capped', 'no_of_ctc_up
```

In [129... df_scaled.head()

```
Out[129...
   YOE_capped  no_of_ctc_update  company_hash_encoded  job_position_encoded
0   -0.130162         -0.505725         -0.485451         -0.361577
1    0.132565         -0.505725         -0.491936          0.454065
2   -0.392890         -0.505725         -0.473777          0.454065
3   -0.655617         -0.505725         -0.084654         -0.368655
4   -0.918345         -0.505725         -0.471183          0.454065
```

Model Building

K-means Clustering

Hierarchical Clustering

```
In [130... def hopkins_statistic(X):
    X = np.array(X) # Ensure X is a numpy array
    d = X.shape[1] # Number of dimensions
    n = len(X) # Number of data points
    m = int(0.1 * n) # Subset size (10% of the data points)

    nbrs = NearestNeighbors(n_neighbors=1).fit(X)
    rand_X = np.random.random((m, d)) * np.amax(X, axis=0)
    u_distances, _ = nbrs.kneighbors(rand_X, 2, return_distance=True)

    w_distances, _ = nbrs.kneighbors(X[np.random.choice(n, m, replace=False)],

    u_distances = u_distances[:, 1]
    w_distances = w_distances[:, 1]

    H = (np.sum(u_distances) / (np.sum(u_distances) + np.sum(w_distances)))
    return H

hopkins_score = hopkins_statistic(df_scaled)
print(f"Hopkins Statistic: {hopkins_score}")
```

Hopkins Statistic: 0.9827602750018937

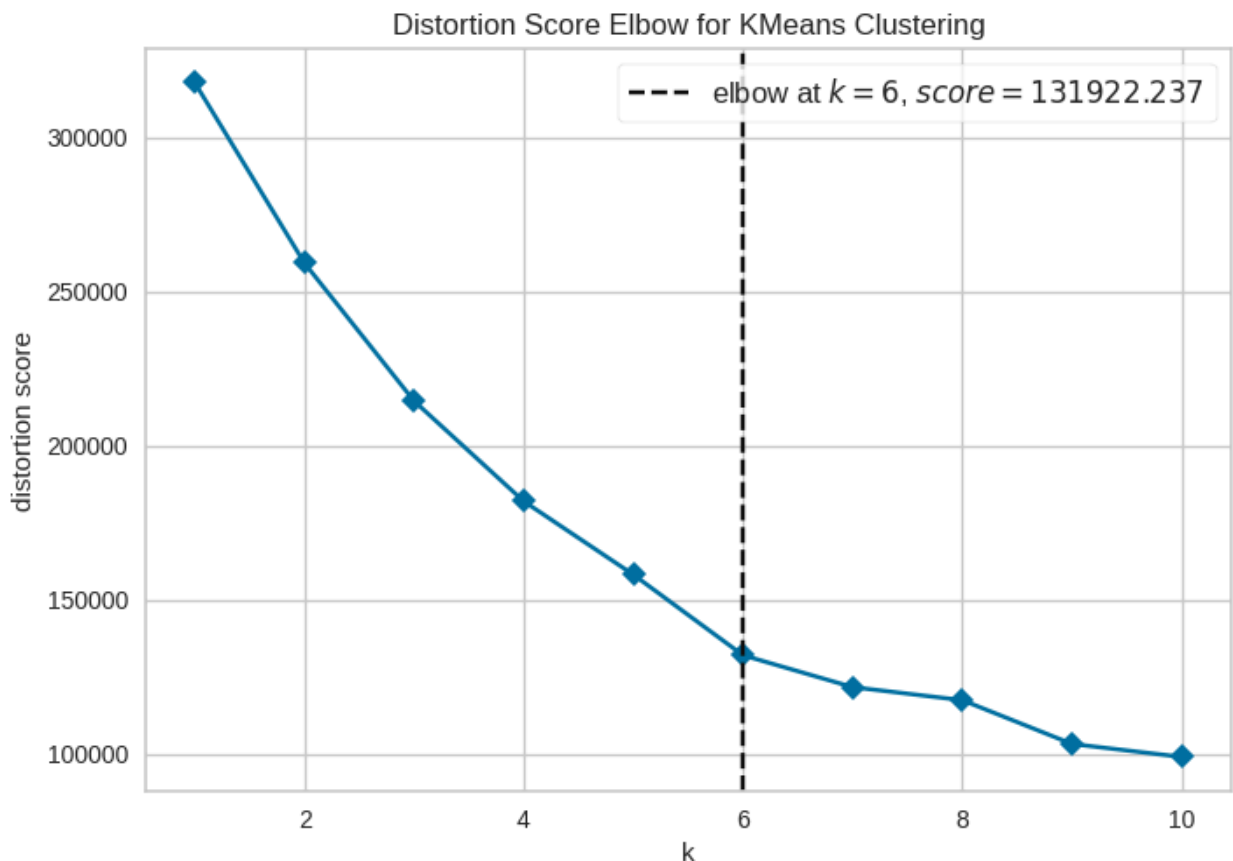
The value is very close to 1, which means that the dataset has a very strong clustering structure. It is likely to form well defined clusters

Elbow Method- To select optimal number of clusters

Inertia

Within Cluster Sum of Squares. This metric measures how tightly the clusters are packed. Lower inertia values indicate better-defined clusters.

```
In [134... model = KMeans()  
# k is range of number of clusters.  
visualizer = KElbowVisualizer(model, k=(1,11), timings= False)  
visualizer.fit(df_scaled)      # Fit data to visualizer  
visualizer.show()
```



```
Out[134... <Axes: title={'center': 'Distortion Score Elbow for KMeans Clustering'}, xlabel='k', ylabel='distortion score'>
```

The elbow point suggests that 5 clusters is a good choice for our data.

This is where the inertia starts to decrease at a slower rate, indicating that additional clusters beyond this point don't significantly improve the clustering quality.

K-Means Clustering

```
In [135... optimal_clusters = 5 # Set the optimal number of clusters as found above
```

```
kmeans = KMeans(n_clusters=optimal_clusters, random_state=42)
kmeans.fit(df_scaled)

# Adding cluster labels to the DataFrame
df5['kmeans_cluster'] = kmeans.labels_
```

In [136... df5.head()

Out[136...

	company_hash	job_position	ctc_capped	YOE_capped	no_of_ctc_update	kme
0	atrgrxnnnt xzaxv	Other	1100000	9	1	
1	ojzwnvwnxw vx	Backend Engineer	2000000	10	1	
2	ngpgutaxv	Backend Engineer	700000	8	1	
3	lubgqsvz wyvot wg	FullStack Engineer	1500000	7	1	
4	vwwtznht ntwyzgrgsj	Backend Engineer	400000	6	1	

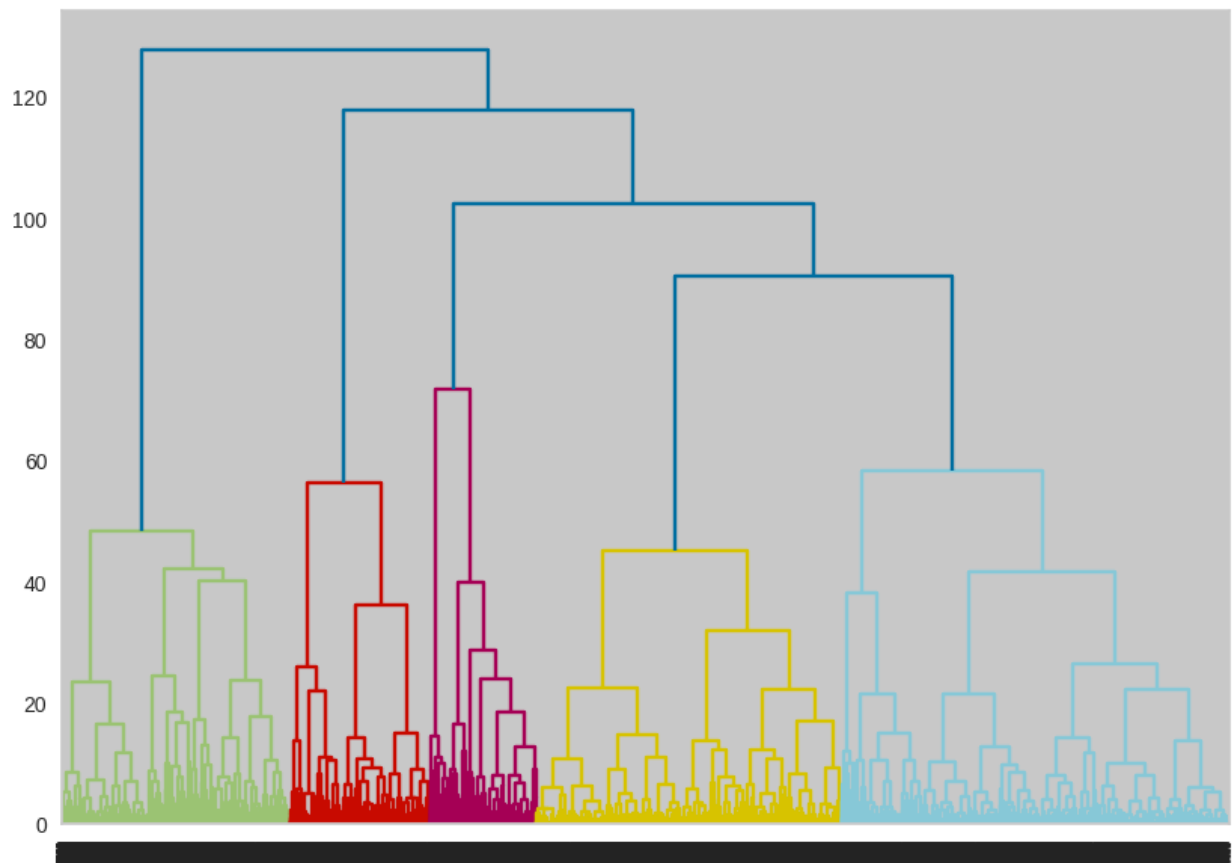
Hierarchical Clustering

In [139...

```
# Sample a subset of the data
df_sampled = df_scaled.sample(n=10000, random_state=42)

# Perform hierarchical clustering
Z = linkage(df_sampled, method='ward')

# Plot the dendrogram
plt.figure(figsize=(10, 7))
dendrogram(Z)
plt.show()
```



Used Representative subset of data to avoid running out of memory.

Dendrogram is showing 5 different colored branches at the end representing 5 clusters

Both Elbow Method and Dendrogram are suggesting 5 clusters for the given dataset

Evaluation of K-means Clustering

Within-Cluster Sum of Squares (WCSS)

The Within-Cluster Sum of Squares (WCSS) is a measure of the compactness of the clusters formed by the K-means algorithm. It represents the sum of squared distances between each data point and its corresponding cluster centroid.

A lower WCSS value indicates tighter clusters, meaning that the data points within each cluster are closer to their respective centroid.

```
In [140... wcss = kmeans.inertia_  
print(f'Within-Cluster Sum of Squares (WCSS): {wcss}')
```

Within-Cluster Sum of Squares (WCSS): 146603.758789699

WCSS Value Consistency: The WCSS value remains consistent at 146603.758789699 for $k=5$. This value represents the total within-cluster variance for the five clusters formed by K-means.

Optimal Number of Clusters:

The elbow method helps identify the optimal number of clusters by plotting WCSS values for different k values and looking for a point where the decrease in WCSS slows down.

If $k=5$ is identified as the elbow point, it suggests that adding more clusters beyond this number does not significantly reduce the WCSS, indicating diminishing returns in terms of cluster compactness.

Between-Cluster Sum of Squares (BCSS)

This value represents the total squared distance between each cluster centroid and the overall mean of the data, weighted by the number of points in each cluster.

A higher BCSS indicates that the cluster centroids are far from the overall mean, suggesting well-separated clusters.

```
In [141]: # Assuming df_scaled is your scaled dataframe
df_scaled_copy = df_scaled.copy()

# Adding cluster labels to the DataFrame
df_scaled_copy['kmeans_cluster'] = kmeans.labels_

# Between-Cluster Sum of Squares (BCSS)
def calculate_bcsc(df, kmeans):
    cluster_centers = kmeans.cluster_centers_
    overall_mean = df.drop(columns='kmeans_cluster').mean(axis=0)
    bcsc = 0
    for i, center in enumerate(cluster_centers):
        size = len(df[df['kmeans_cluster'] == i])
        bcsc += size * np.sum((center - overall_mean) ** 2)
    return bcsc

bcsc = calculate_bcsc(df_scaled_copy, kmeans)
print(f'Between-Cluster Sum of Squares (BCSS): {bcsc}')
```

Between-Cluster Sum of Squares (BCSS): 171159.86747527428

High BCSS and Low WCSS:

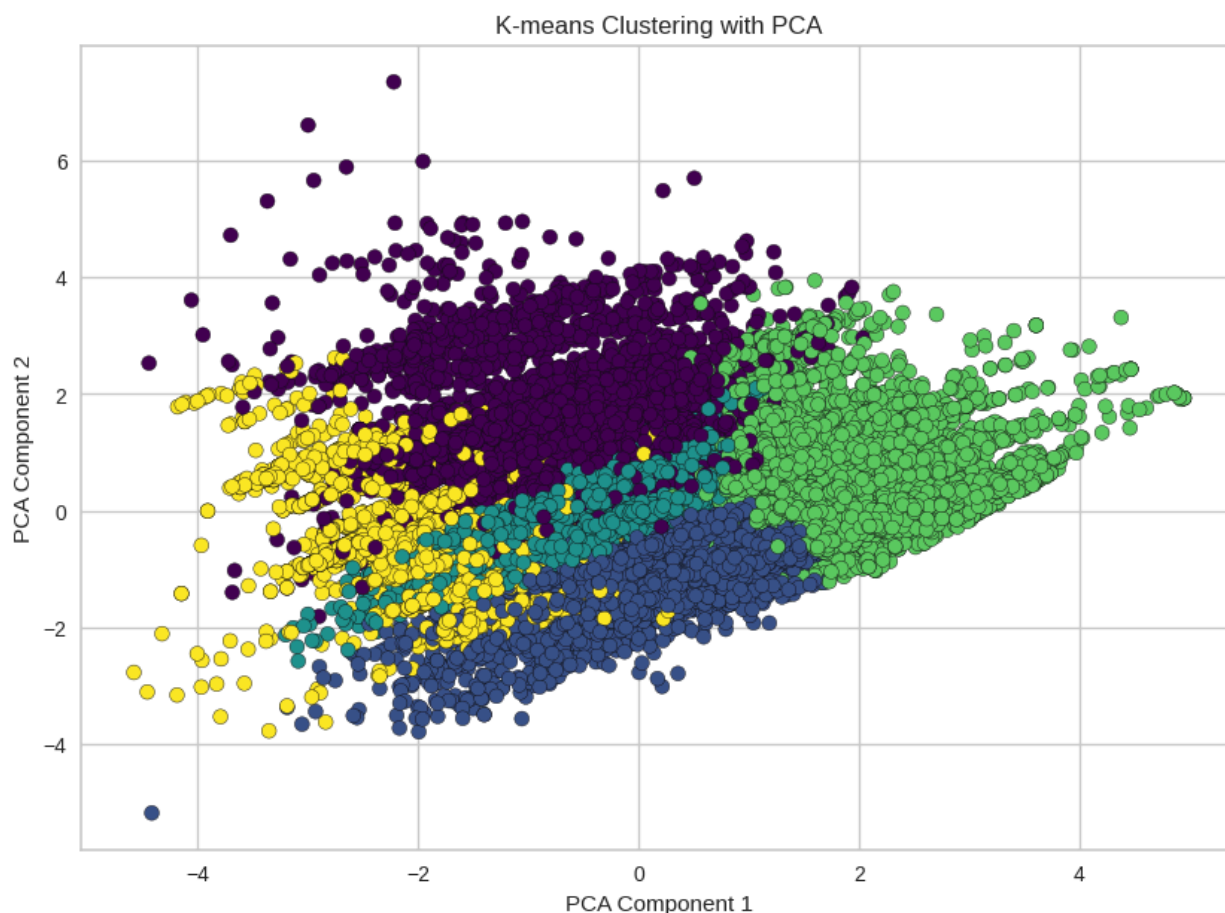
The combination of a relatively high BCSS and a relatively low WCSS is desirable. It means that the clusters are well-separated and compact.

Visual Inspection- PCA

```
In [142... #Visual Inspection using PCA
pca = PCA(n_components=2)
pca_result = pca.fit_transform(df_scaled_copy.drop(columns='kmeans_cluster'))

df_scaled_copy['pca_one'] = pca_result[:, 0]
df_scaled_copy['pca_two'] = pca_result[:, 1]

plt.figure(figsize=(10, 7))
plt.scatter(df_scaled_copy['pca_one'], df_scaled_copy['pca_two'], c=df_scaled_
plt.title('K-means Clustering with PCA')
plt.xlabel('PCA Component 1')
plt.ylabel('PCA Component 2')
plt.show()
```



Recommendations

Scaler has a lot of learners belonging to junior/mid-senior roles and hence should design more courses which will help these learners enhance their skills and move up the career ladder.

Scaler can attract more people to their learning platform by running ads of how Software Development and Data Analyst/Scientist roles get high salary. Scaler should make efforts to pull in more people from Academia (both students and

teachers) to increase their learners base as well as make the students industry ready and employable with high salaries.

The clustering algorithm can be bettered by asking the learners to mention their job position more precisely and specifically instead of just mentioning "Others"

Scaler should also ask the learners to mention their domain of study/work which again will be helpful in improving the clustering algorithm